



Degree Project in Applied and Computational Mathematics

Second cycle, 30 credits

Kolmogorov-Arnold Networks for Explainable Reinforcement Learning

A Symbolic Approach to Function Approximation in
Reinforcement Learning

MICHELE ANTONIO D'ALTO



Kolmogorov-Arnold Networks for Explainable Reinforcement Learning

A Symbolic Approach to Function Approximation in
Reinforcement Learning

Michele Antonio D'Alto

Master's Programme, Applied and Computational
Mathematics

Academic Supervisor: David Kern

Industrial Supervisors: Agustín Valencia, Franco Ruggeri

Examiner: Ozan Öktem

School of Engineering Sciences

Host company: Ericsson AB

Swedish title: Kolmogorov-Arnold Nätverk för förklarbar
Förstärkningsinlärning

Swedish subtitle: Ett symboliskt angreppssätt för
funktionsapproximation inom Förstärkningsinlärning



Abstract

This thesis explores the effectiveness of Kolmogorov-Arnold Networks as interpretable function approximators in a Deep Reinforcement Learning context. We integrate Kolmogorov-Arnold Networks as both actor and critic networks within the Proximal Policy Optimization algorithm and benchmark their performances against Multi-Layer Perceptrons in two environments: CartPole and a Remote Electrical Tilt scenario. By integrating structured pruning, spline regularisation, and symbolic regression, Kolmogorov-Arnold Networks produce compact policies characterised by sparse connectivity and interpretable symbolic expressions.

Kolmogorov-Arnold Networks agents match the performance of Multi-Layer Perceptrons while requiring significantly fewer parameters. Moreover, Kolmogorov-Arnold Networks make the extraction of closed-form symbolic policies possible, accurately approximating the learned behaviour and exposing the decision structure of the agent. In both environments, the symbolic outputs reflected the relevant input variables and control strategies. While Kolmogorov-Arnold Networks introduce additional training overhead, the resulting improvements in transparency and compactness make them a promising alternative for Reinforcement Learning applications where interpretability is essential.

Keywords

Reinforcement Learning, Kolmogorov-Arnold Networks, Interpretability, Symbolic Policies, Proximal Policy Optimization, Radio Access Networks

Sammanfattning

Denna uppsats undersöker effektiviteten hos Kolmogorv-Arnold Nätverk som tolkningsbara funktionsapproximatorer i ett Deep Förstärkningsinlärning-sammanhang. Vi integrerar Kolmogorv-Arnold Nätverk som både aktörs- och kritikernätverk i Proximal Policy Optimization-algoritmen och jämför deras prestanda med Multi-Layer Perceptrons i två miljöer: CartPole och ett Remote Electrical Tilt-scenario. Genom att integrera strukturerad beskärning, spline-regularisering och symbolisk regression producerar Kolmogorv-Arnold Nätverk kompakta policyer som kännetecknas av gles konnektivitet och tolkningsbara symboliska uttryck.

Kolmogorv-Arnold Nätverk-agenter matchar prestandan hos Multi-Layer Perceptrons samtidigt som de kräver betydligt färre parametrar. Dessutom gör Kolmogorv-Arnold Nätverk det möjligt att extrahera symboliska policyer i sluten form, vilket ger en exakt approximation av det inlärda beteendet och avslöjar agentens beslutsstruktur. I båda miljöerna återspeglade de symboliska utdata de relevanta ingångsvariablerna och kontrollstrategierna. Även om Kolmogorv-Arnold Nätverk medför ytterligare utbildningsomkostnader, gör de resulterande förbättringarna i transparens och kompakthet dem till ett lovande alternativ för förstärkningsinlärning-applikationer där tolkningsbarhet är avgörande.

Keywords

Förstärkningsinlärning, Kolmogorov–Arnold Nätverk, Tolkbart,
Symboliska Policies, Proximal Policy Optimization, Radio Access
Networks

Acknowledgments

This thesis marks the end of an intense and meaningful journey, one that I could not have completed alone. I am deeply grateful to the people who supported me along the way, each in their own way, leaving a mark on this work and on me.

To my family, my mother Roberta, my father Vincenzo, and my brother Francesco, thank you for your constant love, strength, and encouragement. No matter the distance, your support has always felt close, and I carry it with me every day. I dedicate this work to my grandfather, Nonno Antonio, whose quiet wisdom and curiosity still guide me. I miss you, and I hope you would be proud.

To Alessia and Gianluca, thank you for walking beside me all these years. Your friendship has been a quiet source of strength and comfort, a reminder of where I come from and of the people who truly know me. Knowing you've always been there has meant more than I can say.

To Kevin, meeting you here changed my experience of moving to Sweden. Your support during moments of uncertainty and your generosity made a real difference when I needed it most. I'm deeply grateful for everything you've shared, your time, your presence, and your friendship.

I am deeply thankful to my supervisors at Ericsson, Agustín and Franco, for offering me the opportunity to work on a project that bridges research and real-world impact. Your mentorship, trust, and insights have shaped both this thesis and my personal growth. I also extend my sincere thanks to my academic supervisor, David, whose guidance challenged me to think more deeply and write with clarity.

To everyone who walked with me during this time, thank you. This journey was never mine alone.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research Questions	3
1.3	Ethics and Environmental Impact	3
2	Background	5
2.1	Kolmogorov-Arnold Networks	5
2.1.1	Kolmogorov-Arnold Representation Theorem	6
2.1.2	Universal Approximation Theorem	9
2.1.3	Constructing Learnable KANs	10
2.1.4	KANs Architecture and Design	14
2.1.5	Interpretability in KANs	16
2.1.6	KANs Workflow	21
2.2	Reinforcement Learning	23
2.2.1	Markov Decision Processes	25
2.2.2	Policy Gradient Methods	27
2.2.3	Proximal Policy Optimization	28
2.3	Interpretability	31
2.3.1	General Taxonomy of Interpretability	32
2.3.2	A Taxonomy for Explainable Reinforcement Learning	33
2.3.3	Interpretability in Kolmogorov-Arnold Networks for Reinforcement Learning	34
3	Methodology	37
3.1	Problem Setting and Objectives	37
3.2	Environments	39
3.2.1	CartPole	39
3.2.2	Antenna Tilt Optimization Environment	40
3.3	Agents architecture	47

3.3.1	KAN-Based Agent	47
3.3.2	MLP Baseline	48
3.3.3	Shared Architecture Considerations	48
3.4	Training	50
3.4.1	Overview	50
3.4.2	PPO Update with Regularisation	50
3.4.3	Full Training Algorithm	51
4	Results	54
4.1	Experiments on the CartPole Environment	55
4.2	Experiments on the RAN Environment	61
5	Discussion	72
5.1	Discussion on the CartPole Environment	72
5.2	Discussion on the RAN Environment	73
6	Conclusion	77
	References	81

List of Figures

1	Decomposition of a multivariate function using the Kolmogorov-Arnold Representation Theorem (KART). From [7].	8
2	Splines and B-splines functions. From [7].	14
3	The SiLU (Sigmoid-Weighted Linear Unit) function.	16
4	Kolmogorov-Arnold Networks (KAN) workflow: from dense training with sparsification to symbolic representation. From [7].	22
5	Reinforcement Learning agent-environment interaction. From [16].	23
6	Deep Reinforcement Learning agent-environment interaction. From [16].	24
7	Proposed XRL taxonomy and its relationship to the RL problem. From [23].	34
8	CartPole environment and state variables. The cart can be pushed left or right, while the agent must account for both the cart's position x and velocity $\dot{x}$, as well as the pole's angle θ and angular velocity $\dot{\theta}$. From [26].	40
9	Schematic illustration of a base station with three sectorized antennas. From [27]	41
10	Reinforcement Learning (RL) loop for Remote Electrical Tilt (RET) control. The agent observes the network state, performs tilt actions, and receives feedback based on updated Key Performance Indicator (KPI)s. From [28].	44
11	RET environment visualisation.	45

12	RET-related effects of tilt changes: (1) coverage hole at cell border, (2) coverage extension, (3) interference on neighbour, and (4) overshooting. From [28].	46
13	Cumulative reward per episode for KAN (blue) and MLP (red). Curves are smoothed using a 50-episode moving average. Shaded areas show standard deviation across five seeds.	57
14	Training time distribution (in minutes) for MLP (red) and KAN (blue) across five random seeds.	57
15	Symbolic policy formula and decision boundary in the $(\theta, \dot{\theta})$ plane.	58
16	Initial dense KAN at initialisation, with randomly initialised splines. All input variables are connected.	59
17	KAN after training with sparsity-inducing regularisation. The model has begun deactivating unnecessary edges and splines.	59
18	Post-pruning: the network retains only a subset of edges. Notably, inputs x and $\dot{x}$ are no longer used, as the model learns to rely on θ and $\dot{\theta}$	60
19	Symbolified KAN model. Red spline functions have been replaced by closed-form expressions.	60
20	Cumulative reward per episode for KAN (blue) and MLP (red). Curves are smoothed using a 50-episode moving average. Shaded areas show standard deviation across five seeds.	64
21	Training time distribution (in minutes) for MLP (red) and KAN (blue) across five random seeds.	64
22	Cumulative reward per episode for KAN before (blue) and after (green) symbolification. Curves are smoothed using a 50-episode moving average. Shaded areas show standard deviation across five seeds.	65
23	Reward distribution comparison between MLP and KAN agents in the RAN environment.	66
24	The ReLU (Rectified Linear Unit) function.	67

25	Symbolic policy formula and decision boundary in the (SISR, RSRP) plane for each value of the tilt angle θ	68
26	Initial dense KAN at initialisation, with randomly initialised splines. All input variables are connected.	69
27	KAN after training with sparsity-inducing regularisation. The model has begun deactivating unnecessary edges and splines.	69
28	Symbolified KAN model. Red spline functions have been replaced by closed-form expressions.	70

List of Tables

1	Common PPO hyperparameters used during CartPole experiments.	55
2	Architecture details for the MLP actor and critic networks for CartPole.	55
3	Architecture details for the KAN actor and critic networks for the CartPole experiments.	56
4	KAN-specific regularisation and pruning parameters for CartPole.	56
5	Mean and standard deviation of training time across five runs.	58
6	Common PPO hyperparameters used during RET experiments.	61
7	MLP architectures used for the RAN environment.	61
8	KAN regularisation hyperparameters tested in the RAN environment.	62
9	KAN actor and critic architecture settings explored in the RAN environment.	62
10	Best MLP architecture (baseline).	63
11	Best KAN hyperparameters for the RAN environment.	63
12	Trainable parameter counts for the selected models.	63
13	Mean and standard deviation of training time across five runs.	65
14	Mean and standard deviation of rewards across evaluation episodes.	66

List of Algorithms

1	KAN-PPO Training Loop	52
---	---------------------------------	----

Acronyms

AI Artificial Intelligence. 1–3, 5

ANN Artificial Neural Networks. 9

BSs Base Stations. 41, 44, 45

DL Deep Learning. 6, 9

DRL Deep Reinforcement Learning. , 1, 23, 24, 38

GAE Generalized Advantage Estimation. 28, 29, 49, 50

GCQ Good Coverage and Quality. 43, 44

KAN Kolmogorov-Arnold Networks. , 2, 3, 5, 6, 8–11, 13–17, 20–22, 24, 25, 28, 34, 35, 37, 39, 40, 47–51, 54–56, 61–65, 73–75, 77–79

KART Kolmogorov-Arnold Representation Theorem. , 5–8, 10, 11, 13, 14, 16

KPI Key Performance Indicator. , 41, 43, 44

MDP Markov Decision Process. 25, 26, 33, 34, 44

ML Machine Learning. 5, 7, 8, 31, 32

MLP Multi-Layer Perceptrons. , 1, 3, 5–7, 9, 10, 14, 16, 28, 40, 47–49, 54–56, 61–65, 73, 77–79

MSE Mean Squared Error. 29

NN Neural Networks. 6, 8, 9, 16

PPO Proximal Policy Optimization. , 2, 24, 28–30, 37, 47, 48, 50, 51, 54, 55, 61, 77

RAN Radio Access Networks. 1, 2, 38–40, 48, 54, 73, 75, 77–79

RET Remote Electrical Tilt. , 2, 3, 41, 44, 45, 61

RL Reinforcement Learning. , 1–3, 9, 23, 25–28, 31, 33, 34, 37, 38, 40, 44, 46, 47, 78

RSRP Reference Signal Received Power. 42–44, 74, 75

SGD Stochastic Gradient Descent. 11, 30

SINR Signal to Interference plus Noise Ratio. 42–45, 74, 75

UAT Universal Approximation Theorem. 5, 6, 9, 10

UEs User Equipments. 41–45, 75

XAI Explainable Artificial Intelligence. 6, 31, 32

XRL Explainable Reinforcement Learning. 2, 33–35, 39

1 Introduction

1.1 Motivation

The rapid evolution of mobile communications drives the development of increasingly intelligent and autonomous Radio Access Networks (RAN). As the industry progresses towards the sixth generation (6G) of wireless technology, networks are expected to operate under highly dynamic conditions, offering enhanced capacity, ultra-low latency, high reliability, and improved energy efficiency. Although the precise specifications of 6G are still under development, it is widely acknowledged that Artificial Intelligence (AI) will be a cornerstone in its architecture, enabling real-time adaptation, autonomous optimisation, and cross-layer orchestration [1].

Within this context, RL has emerged as a promising paradigm for enabling data-driven control in RAN. RL agents can learn directly from interaction with the environment, optimising long-term objectives such as quality of service, load balancing, or coverage, without relying on explicit system models or hand-crafted heuristics. Recent studies have demonstrated the potential of RL to handle complex decision-making tasks such as beam forming [2], and antenna tilt optimisation [3].

However, despite the success of RL in simulated environments, its real-world adoption remains limited. One of the significant barriers is the lack of interpretability of Deep Reinforcement Learning (DRL) models, which are typically based on Multi-Layer Perceptrons (MLP). These black-box architectures make it difficult for network operators and engineers to understand the reasoning behind control decisions, raising concerns about safety, reliability, and compliance.

This issue is becoming increasingly relevant as regulations such as

the European Union’s AI Act¹ emphasize the need for transparency and human oversight in the deployment of AI-based systems, especially in critical infrastructure like telecommunications. The Act promotes the development of explainable and trustworthy AI, requiring models to be understandable by human stakeholders and their decisions to be justifiable.

The field of Explainable Reinforcement Learning (XRL) is gaining traction to address this gap. XRL aims to provide insights into the behaviour of RL agents, either through post-hoc explanations or by designing inherently interpretable models [4]. While post-hoc methods like saliency maps or surrogate models can offer limited insight, they often fall short in reliability and completeness, especially in non-linear and non-stationary dynamics typical of wireless environments.

A promising alternative is to embed interpretability directly into the model architecture. KAN are a recent innovation that replaces fixed non-linear activations in neural networks with learnable spline-based functions. This design offers expressive power and the ability to extract symbolic representations of the learned behaviour, enabling human-readable policies.

In this thesis, we explore the integration of KAN within deep reinforcement learning for RAN control tasks. Specifically, we investigate the use of KAN as actor and critic networks within the Proximal Policy Optimization (PPO) framework, evaluating their effectiveness in delivering high-performing, sparse, and symbolically interpretable policies. By focusing on electrical RET, an important control problem in RAN, we aim to assess whether KANs can help bridge the gap between learning performance and model transparency in realistic wireless network environments.

¹More information can be found in AI Act, Shaping Europe’s digital future - European Union.

1.2 Research Questions

To investigate this, we formulate the following research questions:

- **RQ1:** Can Kolmogorov-Arnold Networks be effectively integrated into policy and value networks for Reinforcement Learning?
- **RQ2:** Do Kolmogorov-Arnold Networks-based Reinforcement Learning agents achieve competitive performance compared to conventional MLP-based agents in realistic network scenarios such as Remote Electrical Tilt?
- **RQ3:** To what extent do Kolmogorov-Arnold Networks enhance the interpretability of Reinforcement Learning policies?
- **RQ4:** What computational trade-offs (*e.g.*, training time, convergence behaviour) are introduced by Kolmogorov-Arnold Networks, and are they acceptable in practice?

These questions guide the theoretical, algorithmic, and experimental components of this work. Our goal is not only to validate KAN as a functional alternative but also to explore their impact on the broader agenda of trustworthy and transparent AI for mobile networks.

1.3 Ethics and Environmental Impact

Integrating interpretable models such as Kolmogorov-Arnold Networks promotes ethical Artificial Intelligence by promoting transparency, auditability, and regulatory compliance in critical infrastructure. By enabling symbolic policies with fewer parameters, these models reduce computational overhead, potentially lowering the energy consumption of deployment. This contributes to more sustainable and responsible Artificial Intelligence systems in future wireless networks.

2 Background

2.1 Kolmogorov-Arnold Networks

Deep learning models were leading the surge of interest in AI and Machine Learning (ML) in recent years, which demonstrated unmatched success across a wide range of domains, from natural language processing and computer vision to scientific computing and control problems. At the core of most of these models lie MLP [5, 6], a class of feed-forward neural networks acclaimed for their ability to approximate complex non-linear functions to an arbitrary degree of precision. Despite their widespread adoption and recognised success, they are often criticised for their lack of interpretability; indeed, they are frequently referred to as *black-box* models, limiting their practical application in fields where understanding the internal decision-making process is as essential as achieving high predictive accuracy.

To address this trade-off, a new neural network architecture, namely the Kolmogorov-Arnold Networks (KAN), has been proposed by Liu et al. [7]. While MLP rely on the Universal Approximation Theorem (UAT) as their mathematical backbone, KAN draw direct inspiration from a well-known result in real analysis, the Kolmogorov-Arnold Representation Theorem [8]. The theorem posits that any multivariate continuous function can be represented as, *i.e.* it is equal to, a finite composition of univariate continuous functions and addition operations. The theoretical implications of this result are powerful: it indicates that high-dimensional functions can, in principle, be decomposed into sums of simpler one-dimensional ones. KAN operationalise this intuition by fixing univariate learnable functions along the edges of the network, rather than using fixed non-linearities at the nodes, as is the case with traditional MLP.

This new architecture has several consequences. First, it eliminates the need for linear weight matrices, as each edge becomes a learnable one-dimensional function, typically parametrised by basis spline class functions. Second, it introduces inherent interpretability: the network now relies on a finite set of univariate functions, which can be easily analysed, visualised, and even symbolically approximated.

In this chapter, we lay out a systematic introduction to KAN, outlining their mathematical foundation, architectural design, and specific distinctive features that make KAN particularly suited to an Explainable Artificial Intelligence (XAI) framework. Many of the concepts presented in this chapter originate from the original work of Liu et al. [7], which introduced the KAN architecture and developed its theoretical and practical advantages over conventional MLP.

We organise the remainder of the chapter as follows. We begin by discussing the Kolmogorov-Arnold Representation Theorem and its implications in the context of function approximation. This is then followed by an analysis of the Universal Approximation Theorem, to outline the traditional role of Multi-Layer Perceptrons within the field of Neural Networks and Deep Learning. We then introduce the KAN architecture in detail, clarifying how activation functions are parametrised and how they support the construction of rich yet transparent representations. We examine how design choices such as activation functions, loss regularisation, and sparsification influence the interpretability of KAN.

2.1.1 Kolmogorov-Arnold Representation Theorem

The Kolmogorov-Arnold Representation Theorem (KART) [8] is a foundational result in fundamental analysis and approximation theory. It was initially proved by Andrey Kolmogorov in 1957 and later refined by his student Vladimir Arnold in 1958, giving the theorem its name. The theorem states that any continuous multivariate function can be decomposed into a superposition of continuous unidimensional functions. This result is closely related to Hilbert’s 13th problem, for which

the KART provides a proof of a constrained version of the problem itself.

Theorem 1 (Kolmogorov-Arnold Representation Theorem). Let $f : [0, 1]^n \rightarrow \mathbb{R}$ be any continuous function. Then there exist continuous univariate functions $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$ and $\varphi_{q,p} : [0, 1] \rightarrow \mathbb{R}$, with $q = 1, \dots, 2n + 1$ and $p = 1, \dots, n$, such that:

$$f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \varphi_{q,p}(x_p) \right) \quad (2.1)$$

This representation decomposes a general multivariate function into a summation of transformed sums of univariate functions, effectively reducing the problem of high-dimensional function approximation to learning a finite number of one-dimensional functions.

Figure 1 provides a pictorial representation of the KART. Each $\varphi_{q,p}$ denotes a univariate activation function applied to the input variables x_1 and x_2 , while each Φ_q represents another univariate function that aggregates the inner summations. The black dots indicate summation nodes, which compute the sum of all incoming inputs.

The implications of this theorem are both profound and subtle. From one perspective, the theorem provides a strong theoretical guarantee: it is possible to characterise any continuous multivariate function using only functions of a single variable and addition; this highlights a promising direction for simplifying the learning of high-dimensional functions. On the other hand, the KART is an existence result, meaning that while it guarantees the existence of functions Φ_q and $\varphi_{q,p}$, it does not provide a constructive method for explicitly determining them. Consequently, the theorem does not offer a direct means of obtaining these functions, limiting its immediate applicability in machine learning contexts.

Thus, for many years, the KART was recognised primarily as a result of theoretical interest, with little applicability in machine learning [9]. Many classical ML methods, including MLP, rely on smooth and dif-

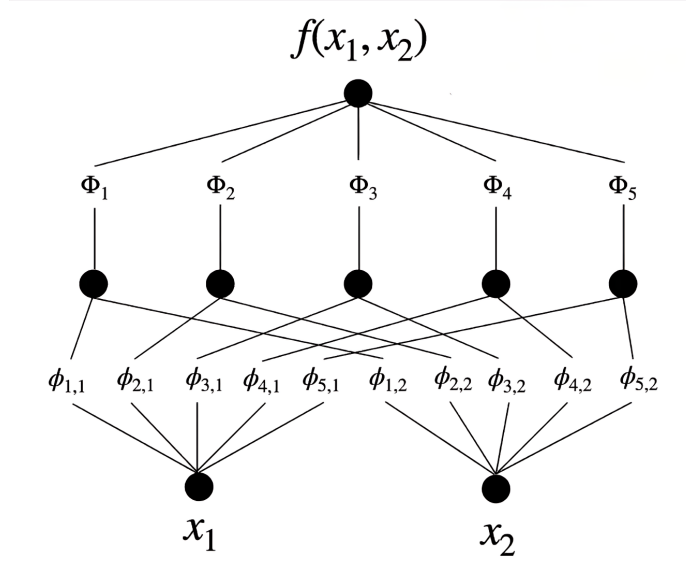


Figure 1: Decomposition of a multivariate function using the KART. From [7].

differentiable activation functions suitable for gradient-based optimisation [10]. The non-constructive and often pathological nature¹ of the functions involved in the original KART limited its influence on Neural Networks (NN) design.

However, recent advances, including the introduction of KAN and its generalisation, have revitalised the interest in KART by proposing architectural strategies that approximate the spirit of the theorem using modern computational tools. In KAN, the univariate functions Φ_q and $\varphi_{q,p}$ are explicitly parametrised as learnable spline-based functions, which are smooth, differentiable, and trainable via backpropagation. This reconceptualisation offers a practical implementation of the KART, laying the foundation for building interpretable and expressive neural networks.

Overall, while the original KART does not provides any explicit guidelines on how to construct the component functions in a ML friendly way, its theoretical insight, that high dimensional functions can be represented via superpositions of one dimensional functions, forms the

¹A pathological function is one constructed to violate standard assumptions, used to test the limits of general mathematical claims. See the Weierstrass function.

mathematical foundation from which KAN are built. This relationship connects classical approximation theory with the current NN design, and plays a key role in the interpretability and expressivity of KAN.

2.1.2 Universal Approximation Theorem

The Universal Approximation Theorem (UAT). [11] is the foundational result in the theory of neural networks. It provides a formal guarantee that MLP, one of the most used types of Artificial Neural Networks (ANN), can approximate any continuous function to arbitrary precision. This theorem supports much of the theoretical confidence in using Deep Learning (DL), from pattern recognition and function regression to control and RL.

Informally, the theorem states that a feedforward neural network with a single hidden layer, containing a finite number of neurons and using a suitable activation function, can approximate any continuous function on a compact subset of $\mathbb{R}^n$, provided the number of hidden units is sufficiently large. More precisely, the classical version of the theorem can be stated as follows:

Theorem 2 (Universal Approximation Theorem). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a continuous function, and let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a nonconstant, bounded, and continuous activation function, for example a sigmoid function. Then, for any $\varepsilon > 0$, there exists a function $\hat{f}(x)$ of the form:

$$\hat{f}(x) = \sum_{i=1}^N a_i \cdot \sigma(w_i^\top x + b_i) \quad (2.2)$$

such that:

$$\sup_{x \in K} |f(x) - \hat{f}(x)| < \varepsilon \quad (2.3)$$

where $K \subset \mathbb{R}^n$ is compact, $a_i \in \mathbb{R}$, $w_i \in \mathbb{R}^n$, and $b_i \in \mathbb{R}$ for all $i = 1, \dots, N$.

In other words, a neural network with enough hidden units can ap-

proximate any continuous function within an arbitrary error margin ε on any compact domain.

This result has been generalised to a wide range of activation functions beyond the sigmoid, including ReLU, $\tanh$, and others, and has been extended to multilayer (deep) networks. While the UAT guarantees expressivity, it does not ensure efficiency or interpretability. In particular, the number of required neurons $N(\varepsilon)$ can become very large as the desired approximation error ε decreases. Moreover, the theorem provides no information about the smoothness or structure of the approximation, and the resulting neural networks can be highly over-parameterised and challenging to analyse.

From a practical point of view, MLP are often referred to as “*black box*” models: while they perform well performance-wise, it is difficult to extract semantically meaningful information from their internal weights and activations. As a result, their usefulness in fields where model transparency is critical can be limited.

This motivates the requests for an alternative method or architecture that combines the expressiveness capabilities of MLP while enabling intrinsic interpretability. This is where KAN come into play: they offer a transparent approach to function approximation, closely aligning with how many physical and engineering systems are expected to behave.

2.1.3 Constructing Learnable KANs

Although both the UAT and the KART provide theoretical guarantees regarding the expressiveness of neural networks, they differ significantly in their practical implications.

On one side, we have the UAT, which is constructive in its formulation, in the sense that it describes a specific architecture, namely, a network with affine transformations followed by fixed activation functions σ , and guarantees that such an architecture, with suitable weights w_i and biases b_i , can approximate a target function. Even though the

theorem does not provide an efficient method for finding these parameters, its formulation is naturally suited for gradient-based optimisation, such as Stochastic Gradient Descent (SGD) and backpropagation [12].

On the other side, the KART is non-constructive. It ensures the existence of functions $\varphi_{q,p}$ and Φ_q such that

$$f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \varphi_{q,p}(x_p) \right).$$

However, the theorem does not explicitly provide instructions on how to find or learn these functions. To overcome this implementation gap, KAN described by Liu et al. [7] introduce a key improvement: the functions involved are parametrised by a family of smooth, local, and learnable functions, namely spline functions. This approach allows us to translate the problem of finding such $\varphi_{q,p}$ and Φ_q into a form compatible with gradient-based optimisation.

2.1.3.1 Splines Functions

To understand why this parametrisation is effective, we begin with the concept of a spline.

Definition 1 (Spline). A spline $S : [a, b] \rightarrow \mathbb{R}$ is a function defined piecewise by polynomial functions. To define a spline on an interval $[a, b]$, we partition the interval into k ordered, disjoint subintervals:

$$\begin{aligned} & [t_i, t_{i+1}], \quad i = 0, \dots, k-1 \\ & [a, b] = \bigcup_{i=0}^{k-1} [t_i, t_{i+1}) \cup [t_k] = [t_0, t_1) \cup [t_1, t_2) \cup \dots \cup [t_{k-1}, t_k) \cup [t_k] \\ & a = t_0 \leq t_1 \leq \dots \leq t_{k-1} \leq t_k = b \end{aligned}$$

The $k+1$ points t_i are called knots, and the vector $\mathbf{t} = (t_0, t_1, \dots, t_k)$ is referred to as the knot vector. On each subinterval $[t_i, t_{i+1}]$, we define

a corresponding polynomial function $P_i : [t_i, t_{i+1}] \rightarrow \mathbb{R}$, and the spline S is defined piecewise as:

$$S(t) = \begin{cases} P_0(t), & \text{if } t_0 \leq t < t_1 \\ P_1(t), & \text{if } t_1 \leq t < t_2 \\ \vdots & \\ P_{k-1}(t), & \text{if } t_{k-1} \leq t \leq t_k \end{cases}$$

If each piece P_i is a polynomial of degree at most n , then the spline S is said to be of order $n + 1$.

An important mathematical trick makes splines even more convenient for learning: any spline function can be written as a linear combination of spline basis functions, functions uniquely defined by their knot values and polynomial degree. Once we have defined the degree of the basis spline and the knot values we are working with, we can describe any spline function as a combination of those basis splines, hence reducing the problem of finding the polynomials in each interval to an optimisation problem where we have to find only the linear combination constants. This approach is well-suited for optimisation methods and can be integrated seamlessly into gradient-based optimisation pipelines.

2.1.3.2 Basis Splines Functions

Basis-splines are piecewise polynomial functions defined over a sequence of knots that provide a flexible yet smooth approximation of arbitrary continuous functions. Their properties, such as local support, differentiability, and ease of optimisation, make them particularly well-suited for learning in neural architectures.

Definition 2 (Basis-spline). A basis-spline of order $n + 1$ is a collection of piecewise polynomial functions $B_{i,p}(t)$ in a variable t . For a given sequence of knots $\mathbf{t} = (t_0, t_1, \dots, t_k)$ there is a unique spline $B_{i,n}(t)$ satisfying:

$$B_{i,n}(t) = \begin{cases} \text{non-zero}, & \text{if } t_i \leq t < t_{i+n+1} \\ 0, & \text{otherwise} \end{cases}$$

B-splines can be constructed recursively, following these steps:

1. Start with B-splines of order $n = 0$:

$$B_{i,0}(t) = \begin{cases} 1, & \text{if } t_i \leq t < t_{i+1} \\ 0, & \text{otherwise} \end{cases}$$

2. The higher $n + 1$ degree B-splines are defined by recursion:

$$B_{i,n} := \frac{t - t_i}{t_{i+n} - t_i} B_{i,n-1}(t) + \frac{t_{i+n+1} - t}{t_{i+n+1} - t_{i+1}} B_{i+1,n-1}(t)$$

Basis-splines are especially useful because they allow any spline function of order n defined on a specified set of knot intervals to be represented as a weighted sum of basis functions. More precisely, if you have a total of G free knot intervals, then the spline function can be expressed as

$$\varphi(x) = \sum_{i=1}^{G+n} c_i B_{i,n}(x) \quad (2.4)$$

where c_i are the learnable coefficients, G is the number of grid intervals, and n is the spline order.

By leveraging splines, KAN operationalise the KART into a form compatible with gradient-based optimisation. This approach not only allows the model to learn from data but also preserves the interpretability advantages given by the one-dimensional structure of the component functions.

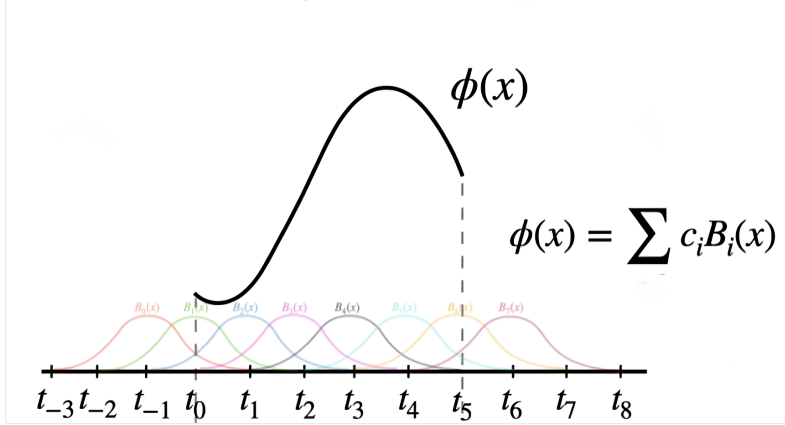


Figure 2: Splines and B-splines functions. From [7].

2.1.4 KANs Architecture and Design

Although the KART guarantees that any multivariate continuous function can be expressed using a composition of univariate functions arranged in a two-layer structure, such a shallow architecture may not be ideal in practice. While the theoretical width of $2n + 1$ hidden units is sufficient for exact representation, real-world learning scenarios involve approximation from data, and often benefit from increased depth and width. As in the case of MLP, deeper architectures can lead to more efficient representations, better generalisation, and more modular feature transformations. For this reason, KAN generalise the original two-layer form into a multilayer architecture in which each layer performs a structured transformation using learned univariate functions.

A KAN layer is defined as follows. Let $x_l \in \mathbb{R}^{n_l}$ denote the vector of node activations at layer l , and let $x_{l+1} \in \mathbb{R}^{n_{l+1}}$ denote the activations at the next layer. Each edge connecting node i in layer l to node j in layer $l + 1$ is associated with a univariate function $\varphi_{l,j,i} : \mathbb{R} \rightarrow \mathbb{R}$. The key idea in KAN is that these edge functions replace both the weights and the non-linearities found in traditional neural networks. Instead of applying a linear map followed by a fixed activation, each input value $x_{l,i}$ is directly transformed by a learnable function before being summed at the next node. The computation at node j in layer $l + 1$ is therefore

given by

$$x_{l+1,j} = \sum_{i=1}^{n_l} \varphi_{l,j,i}(x_{l,i}) \quad (2.5)$$

where each $\varphi_{l,j,i}$ is typically a smooth spline-based function, as discussed in the previous Subsection 2.1.3.2. Each layer thus consists of a matrix of univariate functions, and the overall KAN can be seen as a composition of such function matrices applied layer by layer. Formally, a KAN with $L + 1$ layers and shape $[n_0, n_1, \dots, n_L]$ computes

$$\text{KAN}(\mathbf{x}) = (\Phi_L \circ \dots \circ \Phi_1 \circ \Phi_0)(\mathbf{x}) \quad \mathbf{x} \in \mathbb{R}^{n_0} \quad (2.6)$$

where each Φ_l denotes the function-layer described above.

One challenge with such architectures is initialisation. Since each edge function is parametrised as a spline, and splines alone may be too flexible or start from the function space’s uninformative regions, a residual initialisation scheme is employed. In this scheme, each $\varphi(x)$ is initialised as a weighted combination of a fixed base function and a learnable spline component. A common choice is to use the SiLU function, Sigmoid-Weighted Linear Unit, as the base. The SiLU function, also known as the swish function, is defined as

$$\text{SiLU}(x) = \frac{x}{1 + e^{-x}} \quad (2.7)$$

which can equivalently be written as $x\sigma(x)$, where $\sigma(x)$ is the sigmoid function. This function provides a smooth activation that improves gradient flow during training, as illustrated in the plot in Figure 3.

We can then combine SiLU into our activation functions as follows

$$\varphi(x) = w_b \cdot \text{SiLU}(x) + w_s \cdot \text{spline}(x) \quad (2.8)$$

where w_b is initialised according to Xavier initialisation, $w_b \sim \mathcal{U}[-x, x]$, $x = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}$ and $w_s = 1$. This allows the network to begin with simple, smooth transformations and gradually learn more complex behaviours

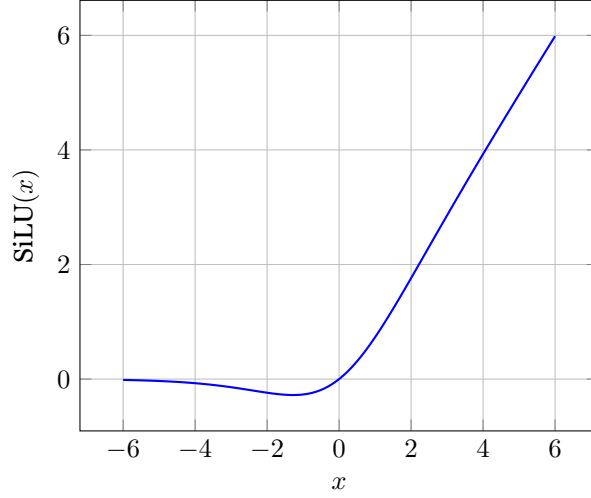


Figure 3: The SiLU (Sigmoid-Weighted Linear Unit) function.

as needed.

In summary, this new KAN architecture generalises the practical application of the KART in a way that is expressive, modular, and compatible with modern training methods and pipelines. By stacking layers of univariate functions, the network can approximate complex functions while maintaining a level of transparency standard MLPs lack. In the next section, we will analyse how these edge functions are computed in practice and how their properties contribute to the interpretability of KAN.

2.1.5 Interpretability in KANs

One of the most convincing rationales for using KAN, rather than MLP, lies in their intrinsic interpretability property. While standard NN achieve state-of-the-art performances at the cost of transparency, KAN are designed with structural properties that make them more suitable for analysis and simplification. The key insight is that non-linearities are introduced into the network through univariate function transformations, making it possible to independently reason about, visualise, and regularise these components. That said, the full interpretability potential of a KAN becomes evident only when certain additional strate-

gies and regularisations are applied during training and post processing.

2.1.5.1 Sparsity and Pruning

Our initial observation is that smaller networks are usually more interpretable than larger ones. When the architecture has fewer nodes per layer or fewer edge connections, the number of components to be analysed and understood is significantly reduced. However, excessive downsizing of the network can mine its expressiveness and ability to approximate complex functions. To address this trade-off, KAN make use of sparsity regularisation terms during training, which allows us to train a larger network and then reduce the number of active components, so that we end up with the minimal size of the network that can solve our task.

The first such regularisation term is an L^1 penalty applied to the outputs of the activation functions. Since each function $\varphi_{i,j}(x)$ contributes to a successor node via summation, a function that remains close to zero across the input distribution has a negligible impact on the model's output. By penalising the L^1 -norm of these activations across training samples, the model is encouraged to suppress functions that are not impactful, effectively pushing them toward being inactive, *i.e.* their representation becomes the zero function, $\phi(x) \approx 0$. This leads to natural sparsification of the architecture: a fraction of the edge functions can become inactive and subsequently can be pruned from the network.

Formally, for an activation function φ evaluated on a batch of N_p inputs $x^{(s)}$, its L^1 norm is defined as:

$$\|\varphi\|_1 \equiv \frac{1}{N_p} \sum_{s=1}^{N_p} |\varphi(x^{(s)})|, \quad (2.9)$$

i.e. its average activation magnitude.

For a KAN layer Φ with n_{in} inputs and n_{out} outputs, the total L^1 norm

is given by:

$$\|\Phi\|_1 \equiv \sum_{i=1}^{n_{\text{in}}} \sum_{j=1}^{n_{\text{out}}} \|\varphi_{i,j}\|_1, \quad (2.10)$$

i.e. the sum of the L^1 norms of the activation functions constituting this layer.

Complementary to sparsifying the network structure, it is also preferable to simplify the form of the learned activation functions themselves. Even if a univariate function is active and influential on the model output, its functional form can be highly nonlinear or oscillatory, which can undermine interpretability. A second regularisation term is introduced to mitigate this: an entropy penalty on the splines' coefficients c_i , as defined in Equation 2.4. In particular, if the spline function is written as a weighted sum of basis-splines, an entropy-inspired measure can encourage the accumulation of weights on fewer basis functions. This should encourage the network to prefer common functional forms, like low-degree polynomials, trigonometric functions, logarithms, and exponentials.

The entropy of a KAN layer Φ is defined as:

$$\mathcal{H}(\Phi) \equiv - \sum_{i=1}^{n_{\text{in}}} \sum_{j=1}^{n_{\text{out}}} \frac{\|\varphi_{i,j}\|_1}{\|\Phi\|_1} \log \left(\frac{\|\varphi_{i,j}\|_1}{\|\Phi\|_1} \right). \quad (2.11)$$

Beyond regularising the outputs of the activation functions, it is also valuable to directly constrain the spline coefficients that define each univariate transformation. Given that each function φ is expressed as a linear combination of B-spline basis functions:

$$\varphi(x) = \sum_{k=1}^K c_k B_{k,n}(x),$$

where c_k denotes the spline coefficient and $B_{k,n}(x)$ is the k -th basis function of degree n , we define two additional regularisation terms. The

first one is the coefficient magnitude penalty, defined as:

$$\mathcal{C}(\Phi) \equiv \sum_{k=1}^K |c_k|.$$

This penalty encourages small coefficients, which in turn yield smoother and less oscillatory functions.

The second one is the coefficient smoothness penalty, defined as:

$$\mathcal{C}_{\text{diff}}(\Phi) \equiv \sum_{k=1}^{K-1} (c_{k+1} - c_k)^2.$$

This term penalises drastic changes between neighbouring coefficients, leading to globally smooth transitions across the input domain.

The total loss function minimised during training can thus be written as:

$$\begin{aligned} \mathcal{L}_{\text{total}} &= \mathcal{L}_{\text{task}} + \lambda \sum_{l=0}^{L-1} (\mu_1 \|\Phi_l\|_1 + \mu_2 \mathcal{H}(\Phi_l) + \mu_3 \mathcal{C}(\Phi_l) + \mu_4 \mathcal{C}_{\text{diff}}(\Phi_l)) \\ &= \mathcal{L}_{\text{task}} + \lambda \mathcal{L}_{\text{reg}} \end{aligned} \quad (2.12)$$

where $\mathcal{L}_{\text{task}}$ is the primary loss (*e.g.*, regression, classification, or reinforcement learning objective), $\|\Phi_l\|_1$ is the L^1 norm over the activations in layer l , $\mathcal{H}(\Phi_l)$ is an entropy-based measure of spline complexity, $\mathcal{C}(\Phi_l)$ is the sum of absolute spline coefficients, and $\mathcal{C}_{\text{diff}}(\Phi_l)$ measures the smoothness of the coefficient profile. The hyperparameters λ , μ_1 , μ_2 , μ_3 , and μ_4 control the relative strength of each regularization component. In practice, the values $\mu_1 = \mu_2 = \mu_3 = \mu_4 = 1$ are commonly used, while λ is tuned to balance task performance against model sparsity and simplicity.

A pruning step can be applied after training the network with these regularisation terms. Any function whose contribution to the output is approximately zero, $\varphi(x) \approx 0$, can be removed. In practice, due to the

noise and stochasticity of training, we are unlikely to obtain activation functions that are exactly zero. Still, their average output magnitude will be orders of magnitude lower than the network output. For this reason, we remove edges whose output remains below a certain threshold τ . Specifically, any spline-based function $\varphi_{i,j}$, associated with the directed edge $(i \rightarrow j)$ in the KAN graph, whose average activation magnitude satisfies

$$\|\varphi_{i,j}\|_1 < \tau \quad (2.13)$$

is removed from the network. This process reduces the model to a sparse computational graph, in which each node receives input from only a small, meaningful number of active components. As a result, the network is not only computationally efficient but also robust to noise and suitable for human inspection and analysis.

2.1.5.2 Symbolification

Beyond pruning, the final and powerful interpretability step is *symbolification*. Because each active transformation in the network is a univariate function defined over a spline basis, it becomes possible to approximate it using a symbolic analytical expression. The goal of this process is to replace a learned spline function φ with a mathematically recognisable form, such as a linear, polynomial, trigonometric, exponential, or logarithmic function, whenever such a match exists.

However, since the inputs and outputs of spline functions can be arbitrarily scaled or shifted during training, directly replacing φ with a symbolic function f may result in a mismatch. To account for this, the symbolification process searches for an affine transformation of the symbolic function that best matches the learned activation. More precisely, we aim to find constants $a, b, c, d \in \mathbb{R}$ such that the following approximation holds over the activation’s domain:

$$y \approx c \cdot f(ax + b) + d$$

where x and y are, respectively, the pre-activation and post-activation

values observed from the trained network during the training phase.

Once (x, y) pairs are obtained, the parameters a, b, c, d are fitted using a combination of grid search and linear regression. Grid search is typically used to explore a discrete set of candidate scalings and shifts (a, b) , while for each such pair, the optimal (c, d) is computed via least-squares regression to minimise the residual error between the transformed symbolic function and the learned spline output.

When a sufficiently accurate fit is found, *i.e.* the coefficient of determination R^2 between $c \cdot f(ax + b) + d$ and y is greater than a predefined threshold, usually 0.9 or 0.95, the learned spline activation φ is replaced with its symbolic approximation $x \mapsto c \cdot f(ax + b) + d$. This replacement offers a way to recover a human-understandable explanation for part of the model’s internal computation.

When applied to the entire sparse KAN, this process yields a network composed of symbolic functions composed through addition and function composition. The final result is a predictive model that is not only performant but also transparent, one in which each transformation can be plotted, understood, and reasoned about analytically.

2.1.6 KANs Workflow

The entire interpretability pipeline described above can be summarised by the typical *KAN workflow*, illustrated in Figure 4.

This process begins with a dense KAN architecture trained with sparsity-inducing regularisation (Step 1), which encourages many activation functions to become inactive. Once training is complete, the network undergoes a pruning step (Step 2), removing edge functions with negligible contribution and yielding a sparse, computationally minimal graph.

The remaining active activation functions are then examined for potential symbolic structure. For each of them, candidate symbolic functions (*e.g.*, sine, squared, exponential) are proposed (Steps 3a-3c). Using a dataset of pre- and post-activation values (x, y) collected from the network, the system then fits affine parameters (a, b, c, d) to match the

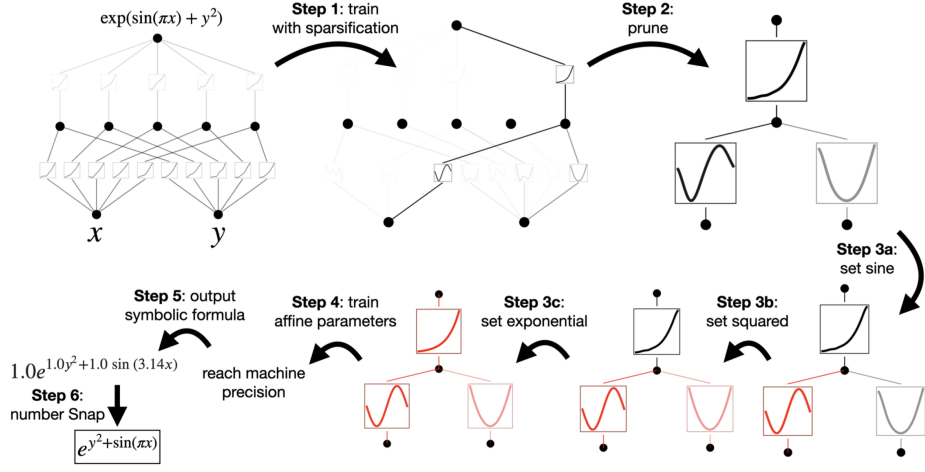


Figure 4: KAN workflow: from dense training with sparsification to symbolic representation. From [7].

learned function to the symbolic one via the transformation

$$y \approx c \cdot f(ax + b) + d$$

(Step 4). Once symbolic matches are found for all active functions, the model outputs the composed symbolic formula (Step 5), which approximates the entire KAN with high fidelity. Finally, an optional simplification step is applied, which rounds constants and refines expressions (e.g., $\sin(3.14x) \rightarrow \sin(\pi x)$), resulting in a clean symbolic expression (Step 6) such as

$$e^{y^2+\sin(\pi x)}.$$

2.2 Reinforcement Learning

Reinforcement Learning [13] is an area of machine learning that characterises how autonomous intelligent agents can learn to make sequential decisions by interacting with an environment. In contrast to supervised learning, where models are trained on input-output pairs, RL relies on a trial-and-error approach: agents explore actions, receive feedback, and attempt to maximise a long-term goal by learning a behaviour. This paradigm is particularly suitable for environments where decisions in the present can impact decisions in the future, and where there is a trade-off between short-term and long-term advantage, such as in robotics [14] and video game playing [15].

In the conventional RL framework, an agent observes the current state of the environment, then selects an action and receives a reward, modelled as a scalar value. After the action is performed, the agent observes the resulting transition, and this process is repeated. Over time, the goal is to learn a policy, *i.e.* a mapping from states to actions, that represents the agent's behaviour such that the cumulative reward is maximised. This interaction is illustrated in Figure 5. The process naturally captures delayed consequences, uncertainty, and strategic planning, making RL a natural framework for learning in dynamic and uncertain environments.

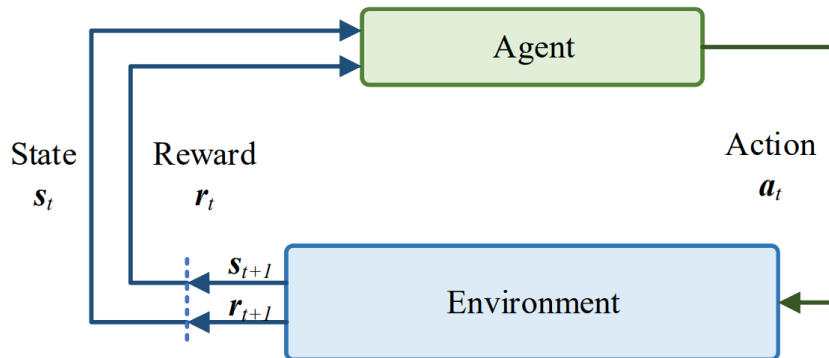


Figure 5: Reinforcement Learning agent-environment interaction. From [16].

Recent developments in DRL have successfully scaled RL to high-

dimensional state and action spaces by using Multi-Layer Perceptrons as function approximators, as illustrated in Figure 6. These methods have enabled agents to learn from raw sensory data, such as images or time series, and to make decisions in real-time environments.

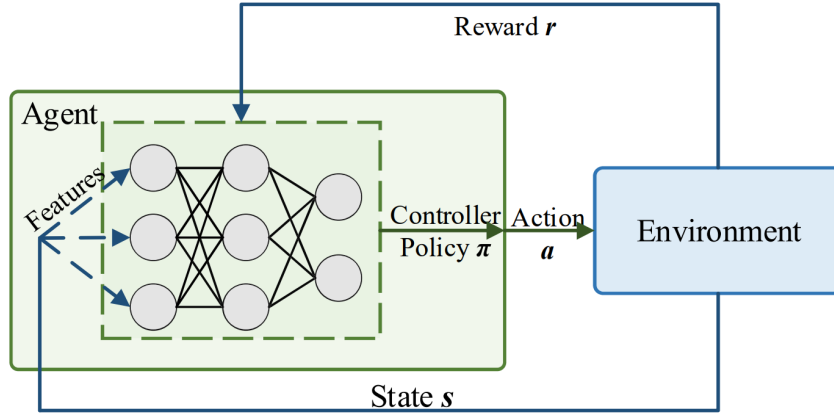


Figure 6: Deep Reinforcement Learning agent-environment interaction. From [16].

In the context of this thesis, DRL is used as the core learning framework to train agents whose decision-making processes are modelled through Kolmogorov-Arnold Networks. Our aim is twofold: to learn an effective policy using KAN as function approximators, and to extract interpretable representations from the learnt models.

Classical DRL algorithms typically rely on approximating the value function, such as Q-learning and its deep variant, Deep Q-learning, whereas this research focuses on policy-gradient methods. To be more precise, we leverage the Proximal Policy Optimization algorithm [17], a state-of-the-art method known for its performance across various complex domains and environments.

The core idea behind PPO is to directly model the policy function by performing constrained updates of the function itself. This avoids the instabilities and scalability limitations of earlier policy methods, such as REINFORCE [18] and Trust Region Policy Optimisation [19]. This approach allows for a natural integration with KAN, facilitating clearer interpretation of the learnt policies.

The rest of this section is organised as follows. We start by reviewing the formal structure of Markov Decision Process (MDP), which underpins most RL problems. We then introduce policy-gradient methods and explain the motivations and the mathematical backbone behind PPO. Finally, we present how it is integrated with KAN in this work.

It is worth noting that much of the theoretical framework presented in this section is inspired by the pioneering work *”Reinforcement Learning: An Introduction”* by Sutton and Barto [13].

2.2.1 Markov Decision Processes

A common assumption in RL is that the environment dynamics can be formalised as a MDP, a mathematical methodology for modelling environments where outcomes are influenced by both the agent’s actions as well as the stochasticity of the dynamics itself.

Formally, a MDP is defined by the tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma),$$

where:

- $\mathcal{S}$ is a (possibly infinite) set of states.
- $\mathcal{A}$ is a set of actions available to the agent.
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the state transition probability function. For any $s, s' \in \mathcal{S}$ and $a \in \mathcal{A}$, the quantity

$$\mathcal{P}(s' \mid s, a) = \mathbb{P}(s_{t+1} = s' \mid s_t = s, a_t = a)$$

specifies the probability of transitioning to state s' after taking action a in state s .

- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is the reward function, where $\mathcal{R}(s, a, s')$ denotes the expected immediate reward received after executing action a in state s .

- $\gamma \in [0, 1)$ is the discount factor, which determines the present value of future rewards by geometrically diminishing their contribution over time.

At each discrete time step t , the agent observes the current state $s_t \in \mathcal{S}$ and selects an action $a_t \in \mathcal{A}$ according to a policy π , which can be deterministic or stochastic. If the policy is stochastic, it is defined as a conditional probability distribution:

$$\pi(a \mid s) = \mathbb{P}(a_t = a \mid s_t = s).$$

Subsequently, the environment responds by transitioning to a new state $s_{t+1} \sim \mathcal{P}(\cdot \mid s_t, a_t)$ and generating a scalar reward $r_t = \mathcal{R}(s_t, a_t)$. This interaction results in a trajectory (or episode) denoted by

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots).$$

The central objective in RL is to identify a policy π that maximises the expected cumulative discounted reward. This objective is formally expressed as

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right],$$

where the expectation is taken over the stochasticity induced by both the policy π and the environment dynamics, as specified by $\mathcal{P}$.

Operationally, maximising $J(\pi)$ requires balancing short-term gains and long-term benefits while operating under uncertainty in state dynamics, as well as mediating between exploration of the environments, useful to gather information, and exploitation, which leverages the current knowledge to maximise expected rewards.

MDP provide a rigorous framework that allows us to reason about these interactions and trade-offs, and most of the RL algorithms and environments, including the one used in this thesis, build upon this formulation.

2.2.2 Policy Gradient Methods

While many classical RL algorithms focus on estimating value functions, *policy gradient methods* aim to directly optimise the parameters of a policy in order to maximise expected return.

In policy gradient methods, the agent's policy $\pi_\theta(a \mid s)$ is a differentiable function parametrised by $\theta \in \mathbb{R}^d$. The goal is to find the parameters θ that maximise the expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]. \quad (2.14)$$

This optimisation problem is solved by computing the gradient $\nabla_\theta J(\theta)$ and using gradient ascent. The central result that enables this approach is the *Policy Gradient Theorem*, which provides an expression for the gradient of the expected return with respect to the policy parameters.

Theorem 3 (Policy Gradient Theorem). Let $\pi_\theta(a \mid s)$ be a differentiable policy. Then, under suitable regularity conditions, the gradient of the expected return is given by:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t \mid s_t) Q^{\pi_\theta}(s_t, a_t)], \quad (2.15)$$

where $Q^{\pi_\theta}(s, a)$ is the action-value function under policy π_θ .

This theorem allows the use of stochastic gradient ascent: samples (s_t, a_t) are drawn from trajectories generated by the current policy, and the gradient estimate is computed using Monte Carlo methods.

In practice, using $Q^{\pi_\theta}(s, a)$ directly is often infeasible due to its high variance and the difficulty of estimating it accurately. Instead, several variants of policy gradient methods use the *advantage function*:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s), \quad (2.16)$$

which measures how much better an action is compared to the average performance at that state.

Using the advantage function, the policy gradient becomes:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi_{\theta}}(s_t, a_t)]. \quad (2.17)$$

To further reduce variance, a baseline function (typically the value function $V^{\pi}(s)$) is subtracted from the return without introducing bias. These modifications improve the stability and efficiency of learning, and are foundational to advanced policy-gradient algorithms such as Proximal Policy Optimization.

In the next section, we present the Proximal Policy Optimization algorithm in detail, emphasising its motivation, mathematical formulation, and practical advantages.

2.2.3 Proximal Policy Optimization

Proximal Policy Optimization (PPO) [17] is a state-of-the-art policy-gradient RL algorithm that combines the stability of trust-region methods with the scalability of policy-gradient approaches. It aims to improve the policy in a way that avoids large, destabilising updates, which are a common issue in vanilla policy-gradient methods. PPO achieves this by introducing a clipped surrogate objective that limits the deviation of the new policy from the old one during each update.

In this thesis, we implement PPO for discrete action spaces using a modular actor-critic architecture. The actor is a neural policy network parametrising a categorical distribution over actions, while the critic estimates the state-value function $V^{\pi}(s)$. Both networks can be instantiated as either standard MLP or KAN, or a mix of both.

2.2.3.1 Advantage Estimation via GAE

To reduce variance and incorporate temporal information in the policy-gradient, we adopt Generalized Advantage Estimation (GAE) [20]. Given a trajectory of states, actions, rewards, and values, the temporal-difference

residual δ_t is computed as:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad (2.18)$$

and the GAE advantage is recursively defined as:

$$\hat{A}_t = \delta_t + \gamma \lambda (1 - d_t) \hat{A}_{t+1}, \quad (2.19)$$

where d_t is the episode termination flag, and $\lambda \in [0, 1]$ is a bias-variance trade-off parameter. The final returns used for value learning are computed as:

$$\hat{R}_t = \hat{A}_t + V(s_t). \quad (2.20)$$

2.2.3.2 Clipped Surrogate Objective

The innovation of PPO is a surrogate loss function that penalises policy updates that move too far from the previous policy. Let π_θ be the current policy and π_{old} be the behaviour policy used to collect data. The objective is defined as:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (2.21)$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)} \quad (2.22)$$

is the importance sampling ratio, and ϵ is the clip parameter. This loss ensures that the new policy does not deviate too aggressively from the old one, thereby improving stability.

2.2.3.3 Value and Entropy Losses

The value function is trained to minimise the Mean Squared Error (MSE) between predicted values and target returns:

$$\mathcal{L}^{\text{VF}} = \frac{1}{2} \mathbb{E}_t \left[(V(s_t) - \hat{R}_t)^2 \right]. \quad (2.23)$$

To encourage exploration, an entropy bonus is added to the policy loss:

$$\mathcal{L}^{\text{ENT}} = \mathbb{E}_t [\mathcal{H}(\pi(\cdot \mid s_t))], \quad (2.24)$$

where $\mathcal{H}$ denotes the entropy of the policy distribution.

2.2.3.4 Total PPO Loss

The overall objective optimised during PPO training combines the clipped surrogate policy loss, a value function regression term, and an entropy bonus to encourage exploration. The complete loss function is defined as:

$$\mathcal{L}_{\text{PPO}} = \mathcal{L}^{\text{CLIP}} + c_v \cdot \mathcal{L}^{\text{VF}} - \beta \cdot \mathcal{L}^{\text{ENT}}, \quad (2.25)$$

where:

- $\mathcal{L}^{\text{CLIP}}$ is the clipped policy loss that constrains updates to the policy,
- $\mathcal{L}^{\text{VF}}$ is the mean squared error loss between predicted values and returns,
- $\mathcal{L}^{\text{ENT}}$ is the entropy bonus promoting policy stochasticity.

The coefficients c_v and β control the relative importance of the value loss and entropy bonus, respectively. This composite objective is minimised using SGD over multiple epochs and minibatches. The clipping mechanism stabilises training by preventing destructive policy updates, while the entropy term ensures continued exploration throughout the learning process.

2.3 Interpretability

With the rise in complexity of machine learning models, uncovering the reasoning behind their outputs becomes more difficult. Moreover, the RL framework introduces an additional layer of complexity, where agents operate over time and learn while interacting with a stochastic environment. Whereas supervised learning models learn static maps between input and output, RL agents take sequential decisions and optimise long-term rewards, relying on complex policies often opaque even to domain experts. This lack of transparency in critical domains, such as healthcare, autonomous systems, and telecommunication, presents a significant obstacle to trust, debugging and adoption.

Despite the central role of interpretability in the ML community, there is no uniform accepted definition of the term itself. The literature is fragmented, using various similar but distinct terms, such as interpretability, transparency, and explainability, often interchangeably. This ambiguity originates from the absence of a formal or universally accepted definition and a subjective usage: different authors make subtle distinctions without defining a common framework.

To contextualise our approach, we adopt the perspective introduced by Puiutta and Veith [4] and start by examining the linguistic definitions of the words interpretation and explanation. While both convey meaning, explanation focuses on making something easier to understand, often by offering reasons or causes. In contrast, interpretation also involves choosing how to convey that information meaningfully and clearly. This distinction is central in Explainable Artificial Intelligence (XAI), where the goal is not only to extract the right information from a given model, but also to present it in a comprehensible form to humans. Consequently, we adopt the following definition for this thesis, as stated by Puiutta and Veith. [4, p. 5]:

Interpretability is the ability not only to extract or generate explanations for the decisions of a model, but also to present this information in a way that is understandable by human

(non-expert) users, to enable them to anticipate and predict the model's behaviour.

2.3.1 General Taxonomy of Interpretability

Interpretability in ML is commonly categorised along two key dimensions: the *timing* of the explanation, intrinsic vs. post-hoc, and its *target*, local vs. global. This classification, established by Puiutta and Veith [4], provides a theoretical foundation for understanding the diverse methods and scopes used in XAI.

The contrast between intrinsic and post-hoc interpretability describes *when* the explanation is obtained. In the case of intrinsic interpretability, we refer to models that are intrinsically explainable by design, through architectural design choices. Linear regression, decision trees and rule-based models are all examples of intrinsic models. The inference process in these models inherently generates interpretable explanations. Nevertheless, these models typically achieve interpretability through architectural simplicity, which limits their predictive performance and, consequently, restricts their applicability to complex problems. On the other hand, post-hoc interpretability refers to methods that generate explanations after our model is trained, and in fact, these methods are model agnostic. Here we find methods like saliency maps, SHAP (SHapley Additive exPlanations) [21] and LIME (Local Interpretable Model-agnostic Explanations) [22]. These methods can, in principle, be applied to black-box models, but their truthfulness in generating explanations is not always guaranteed.

Along the second axis, this taxonomy differentiates between *local* and *global* explanations. As their names suggest, their goal is to provide either instance-specific explanations (local) or general overall ones (global). Local explanations outline why the underlying model made a specific prediction for a particular instance. Global explanations, instead, attempt to generalise the overall behaviour of the model, trying to understand the logic or pattern that applies to all the input space. Local explanations improve trust in specific predictions. Global inter-

pretability, in turn, supports system-wide comprehension and facilitates auditing.

This taxonomy, though foundational, is not capturing the additional complexities that arise in RL tasks, where other dynamics and trade-offs are encountered.

2.3.2 A Taxonomy for Explainable Reinforcement Learning

RL incorporates a unique layer of complexity: here we are computing a map between input states and output actions, and in these mappings are contained not only the environment stochastic dynamics, but also all the trade-offs typical of RL, namely exploration-exploitation and short-term vs long-term rewards. Consequently, the taxonomy introduced in section 2.3.1 cannot account for the temporal stochastic dynamic typical of RL policies.

To address this gap, Milani et al. [23] propose a taxonomy specifically tailored to XRL, identifying three collectively exhaustive interpretability classes:

- Feature importance explanations, emphasise the most influential features that shaped a particular decision of our agent. These explanations are local, explaining why an agent took action a_t in state s_t by analysing which component of s_t led to that decision.
- Learning process and MDP explanations, focus more on the behaviour of the environment and which specific dynamics and/or instances shaped the overall agent behaviour. This involves emphasising influential past experiences involving rewards, decisions and transitions. These explanations can be either local or global, depending on whether they are focusing on the overall trend of the agent or on specific instances.
- Policy-level explanations offer insight into the long-term behaviour of the agent. Because these explanations are global ones, they should exclude details for the sake of interpretability.

Figure 7 illustrates the explained triad of explainability dimensions in XRL, feature importance, learning process and MDP, and policy-level explanations described above. This factorisation reveals that understanding a RL requires not only to reason about individual actions, but also to understand the overall goal of the agent and the dynamic itself. Hence, explainability in RL must operate at multiple semantic levels.

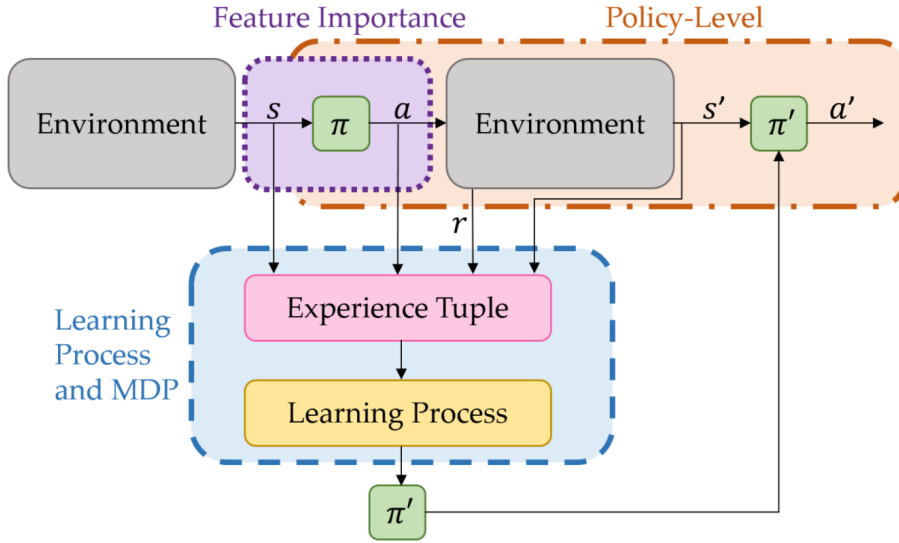


Figure 7: Proposed XRL taxonomy and its relationship to the RL problem. From [23].

2.3.3 Interpretability in Kolmogorov-Arnold Networks for Reinforcement Learning

Kolmogorov-Arnold Networks offer an architecture particularly well-suited for XRL. The design of KAN, based on compositional and hierarchical structure, supports interpretability at both structural and functional levels. This section analyses which elements of the interpretability taxonomies KAN support, highlighting both their contributions and limitations.

Once the network is pruned and sparse, each edge function can be

symbolified, *i.e.* approximated by symbolic regression. This replaces splines with a known, closed-form expression. These expressions provide not only a post-hoc interpretation of the network’s decisions, but also a global picture of the policy’s long-term behaviour and goal, highlighting the most influential input states in the overall decisions and specific subregions of interest. This makes it possible to reason analytically on the agent’s behaviour, and can be used to analyse particular decisions and behaviour in borderline situations, for instance.

Referring to the triad in the XRL taxonomy, KAN support *feature importance* explanations by enabling straightforward identification of the contribution of specific features to a given decision. Moreover, the impact of each input on the policy’s output can be assessed directly.

KAN also excel in *policy-level* explanations, as they allow the extraction of symbolic representations of the value function or policy. These symbolic formulas provide both a global summary of the agent’s behavior, highlighting which features most strongly influence overall decision and a means of analysing behavior in specific sub-regions of interest, particularly in borderline situations, by inspecting which features are most influential and which ones may drive the agent toward a different decision when slightly perturbed.

Finally, KAN can be used to understand what drives the *learning process*, in particular, we can see how activation functions evolve during training, starting from a dense architecture towards and see which features gets flatten out or removed, hence highlighting how the agent’s strategy matures. This visibility into the learning dynamics is rarely accessible in conventional architectures and offers a unique opportunity to interpret how the model discards irrelevant information.

3 Methodology

3.1 Problem Setting and Objectives

This chapter formally defines the research problem and outlines the objectives addressed in this work. Building upon the prior theoretical and architectural foundation of Kolmogorov-Arnold Networks, the focus is now on integrating this architecture into a Reinforcement Learning framework in a way that explicitly promotes interpretability.

Despite the fact that Kich et al. [24] already applied KAN with Proximal Policy Optimization, their focus was on performance metrics, and did not attempt to recover, interpret or analyse the resulting policy. Conversely, our central aim is to leverage the peculiar spline-based architecture of KAN to extract and analyse meaningful explanations of an agent’s model behaviour. This approach extends beyond raw performances, with the goal of obtaining interpretable, and potentially symbolic, policies that can improve trustworthiness and enhance human-in-the-loop decision making in RL.

The main goal of our work is to integrate KAN with policy-gradient methods, specifically Proximal Policy Optimization, and evaluate whether this combination can yield interpretable policies without compromising learning performance.

This work introduces a KAN-based PPO training pipeline that incorporates:

- A sparsity regularisation scheme based on L^1 and entropy penalties over learned spline activations, as discussed in Subsection 2.1.5.1;
- Periodic pruning to eliminate inactive spline functions and reduce architectural complexity;

- Post-pruning *symbolification*, in which active spline transformations are approximated with closed-form expressions via symbolic regression, as discussed in Subsection 2.1.5.2.

Together, these steps are designed to produce compact and human-interpretable policies, where each transformation can be visualised in isolation and the overall policy can be understood as a whole. The resulting agent serves as a link between DRL and interpretability.

To assess the effectiveness of our approach, experiments are first conducted in the CartPole environment, a well-known RL and control benchmark environment. This setting allows for qualitative examination of policy behaviours, serving as a reliable step for debugging.

The intended final application is the control and optimisation of RAN, where interpretable RL agents could support autonomous decision-making in dynamic and high-stakes telecommunication systems.

3.2 Environments

To assess the premises and goals of KAN in the context of XRL, we consider two different environments which differ in complexity and task goal. The first scenario is a standard and well-known environment, namely CartPole, which will help us conduct a preliminary analysis and validate our training pipeline. The second is a real-world-inspired simulation environment, designed to mimic a Radio Access Networks scenario, representing the intended deployment domain for interpretable decision-making.

3.2.1 CartPole

Cartpole is a classic benchmark, originally introduced by Barto, Sutton, and Anderson in their seminal work “*Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems*” [25]. The principal use of this environment is to evaluate and compare learning algorithms in solving a classic control problem.

The task involves balancing a pole mounted on a cart that moves along the horizontal axis. At each time step, the agent must choose between pushing the cart to the left or right, aiming to prevent the pole from falling. The environment is defined by a four dimensional state space:

- Cart position x , representing the horizontal displacement of the cart from the centre of the track,
- Cart velocity $\dot{x}$, indicating how fast the cart is moving,
- Pole angle θ , measuring the deviation of the pole from the vertical axis,
- Pole angular velocity $\dot{\theta}$, representing the rate of change of the pole angle.

Figure 8 provides a visual illustration of these quantities and the dynamics of the system. The agents receives a reward of +1 for every time

step during which the pole is less than $|\theta_{\max}| = 0.412$ radians and the cart position stays within $|x_{\max}| = 2.4$ units. As anticipated before, the action space is discrete, consisting of two actions: pushing left or pushing right.

The environment is accessible through the `gymnasium` package via the standard interface `gymnasium.make("CartPole-v1")`.

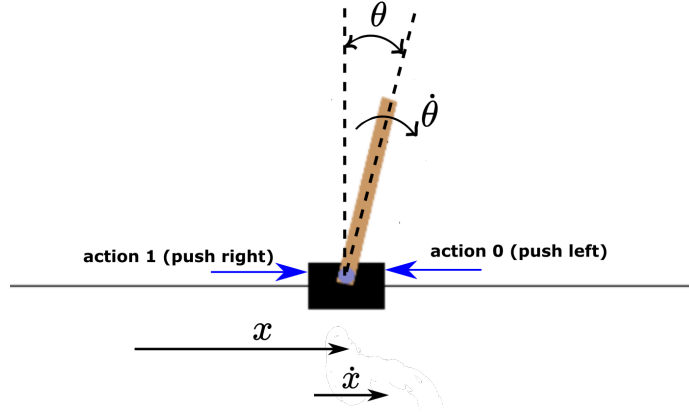


Figure 8: CartPole environment and state variables. The cart can be pushed left or right, while the agent must account for both the cart’s position x and velocity $\dot{x}$, as well as the pole’s angle θ and angular velocity $\dot{\theta}$. From [26].

CartPole is useful in the context of this thesis as it allows us to validate whether a KAN-based agent can effectively replace standard MLP-based agents, improving performance from an explainability perspective without sacrificing expressiveness.

3.2.2 Antenna Tilt Optimization Environment

The second environment considered in this thesis is a proprietary simulation platform developed at Ericsson Research, designed to emulate the dynamics of a real-world RAN in a controlled and reproducible setting. While fully simulated, the environment incorporates realistic propagation models, user distributions, building layouts, and interference patterns representative of modern urban deployments. This makes it an ideal testbed for investigating intelligent control solutions such as RL. In particular, we focus on the problem of antenna tilt op-

timisation, a critical parameter that influences coverage, interference, and overall network quality. Unlike CartPole, which serves as a simplified control benchmark, this environment reflects the complexities and constraints of real mobile networks, including partial observability, agent coordination, and operational safety requirements.

Across today's cellular infrastructures, antenna tilt plays a crucial role in determining both coverage and capacity on the network itself. The tilt angle, particularly the RET one, controls the direction of the antenna's main lobe, influencing how signal power is distributed across the geographical area of interest.

In a typical scenario, a Base Stations (BSs) consists of three antennas, each serving a different sector of the surrounding area of approximately 120° . Adjusting the tilt of these antennas can optimise the signal quality and coverage experienced by users in the network. Figure 9 schematically depicts this configuration.

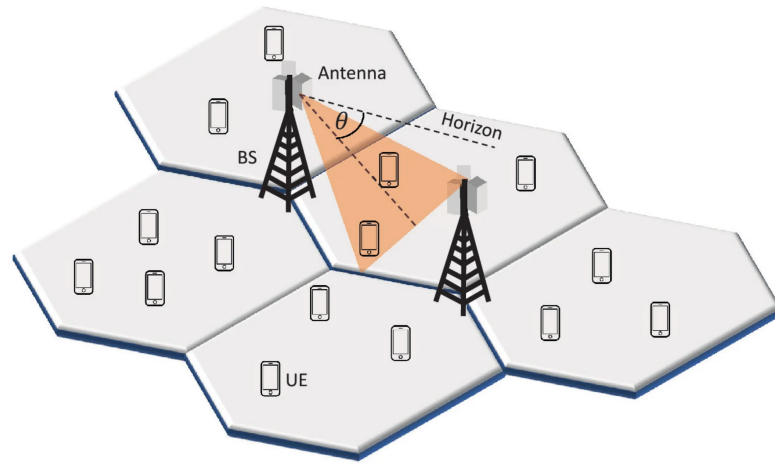


Figure 9: Schematic illustration of a base station with three sectorized antennas. From [27]

Each base station aims to serve User Equipments (UEs) within its sectors, and the main KPI for this experiment is the proportion of UEs experiencing good coverage and good signal quality. The optimisation problem thus becomes: how to control the tilt angle of each antenna to maximise this performance metric over time, while minimising cross-

sector interference and ensuring that the telecommunication network is stable.

State and Action Space

The state space observed by each agent (antenna) consists of three variables that are measurable for each antenna sector:

- AvgRSRP: the average Reference Signal Received Power (RSRP) received by UEs within the sector. RSRP measures the received signal strength and is a key indicator of *coverage*. It is computed as:

$$\overline{\text{RSRP}}(\theta) = \frac{1}{U} \sum_{i=1}^U \text{RSRP}_i(\theta), \quad (3.1)$$

where U is the number of UEs in the sector and θ is the current antenna tilt angle.

- AvgSINR: The average Signal to Interference plus Noise Ratio (SINR) for UEs served by the sector, defined as:

$$\overline{\text{SINR}}(\theta) = \frac{1}{U} \sum_{i=1}^U \text{SINR}_i(\theta), \quad (3.2)$$

where SINR captures the signal quality in the presence of noise and interference from neighbouring sectors.

- Tilt: The current electrical downtilt angle of the antenna, discretised in integer degrees within the interval $[0, 15]$. This angle θ directly influences both RSRP and SINR through the vertical beam pattern of the antenna.

To ensure scale invariance and facilitate learning, the average RSRP and SINR values are normalised using standard score normalisation, z -score. Specifically, we assume:

$$\text{RSRP} \sim \mathcal{N}(-103.0, 23.0^2), \quad \text{SINR} \sim \mathcal{N}(6.25, 2.52^2).$$

In addition, the angular feature θ , which takes integer values in the range $[0, 15]$ degrees, is normalized via min-max scaling to the interval $[-1, 1]$.

These transformations ensure that all features are on comparable scales, centered around zero and bounded where appropriate, thereby promoting numerical stability and efficient learning during model training.

The agent's action space consists of three discrete actions:

$$a \in \mathcal{A} = \{-\delta, 0, +\delta\},$$

representing a decrease, no change, or an increase of the tilt angle by one unit. This incremental policy helps maintain network stability.

Reward Function

The environment is driven by a proprietary KPI referred to as Good Coverage and Quality (GCQ), defined as the proportion of UEs within the sector that simultaneously meet two signal criteria:

- Good Coverage: RSRP exceeds a predefined threshold,

$$\text{RSRP}^+ = -108.0$$

- Good Quality: SINR exceeds a predefined threshold,

$$\text{SINR}^+ = 3.0$$

Both these thresholds have been suggested by network experts.

Thus, the reward received by the agent is a real number in the range $[0, 1]$, indicating the fraction of well-served users. This KPI offers a natural incentive to improve both signal reach and quality, reflecting the trade-off of this tilt optimisation problem.

Summary

To formally summarise the environment from a RL perspective, we define the components of the MDP as follows:

$$\begin{aligned}\mathcal{S} &= \{ [\overline{\text{RSRP}}, \overline{\text{SINR}}, \overline{\theta}] \}, \\ \mathcal{A} &= \{-1, 0, +1\}, \\ \mathcal{R} &\in [0, 1].\end{aligned}$$

Here, the state space $\mathcal{S}$ includes the normalised average RSRP and SINR, as well as the current electrical downtilt angle θ . The action space $\mathcal{A}$ consists of discrete incremental adjustments to the tilt: decrease, no change, or increase. The reward $\mathcal{R}$, represented by the GCQ KPI, measures the proportion of UEs experiencing both strong signal power and satisfactory signal quality.

The typical RL loop applied to the RET control problem is summarised in Figure 10. At each timestep, the agent observes the current network state, selects a tilt action (up, down, or stay), and receives a reward based on the resulting changes in the GCQ KPI. The simulator then transitions to a new state, closing the loop.

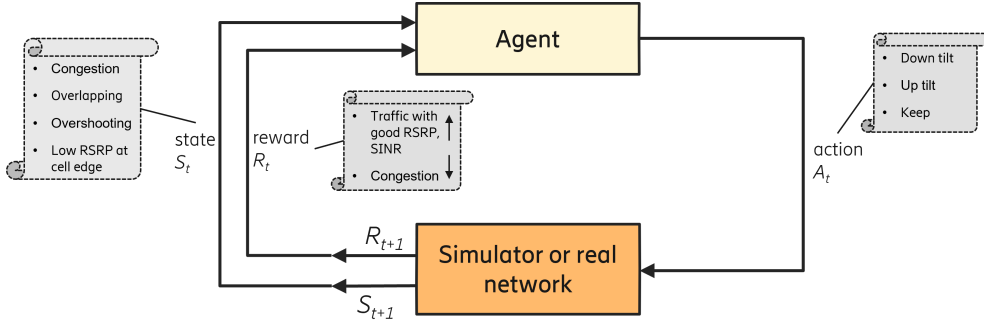


Figure 10: RL loop for RET control. The agent observes the network state, performs tilt actions, and receives feedback based on updated KPIs. From [28].

The Figure 11 below provides a visual representation of a simulated instance of the Ericsson RET environment. In Figure 11a, green stars represent the positions of theBSs, each equipped with independently

controlled antennas to serve three complementary sectors, as shown in Figure 11b. Blue dots correspond to user equipments UEs, which are uniformly distributed throughout the environment and classified as either indoor or outdoor users. This distinction depends on whether the UEs is located and will result in different data requirements. This layout is designed to resemble urban topology, making tilt optimisation more realistic.

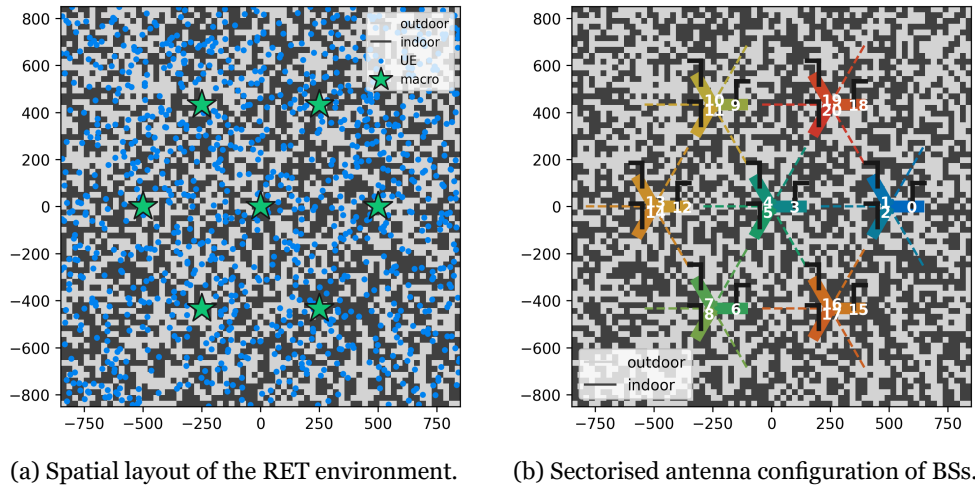


Figure 11: RET environment visualisation.

What makes the RET control problem particularly challenging is the dynamic and interconnected nature of the environment. At the start of each training episode, the initial tilt angle of every antenna is randomly initialised, leading to a wide variety of network conditions. As agents explore the state space and adjust their tilt angles to approach the optimal configuration θ , several undesirable behaviours may emerge, as illustrated in Figure 12.

If the tilt is too steep, the antenna may fail to cover distant users, resulting in a coverage hole at the cell border (1). Increasing the tilt can mitigate this issue by extending the coverage (2). However, if the tilt becomes too shallow, the signal may extend into neighbouring cells, causing a degradation of SINR (3). In more severe cases, the beam may overshoot entirely, leading to unintended coverage far beyond the target area, potentially degrading network performance (4). This makes

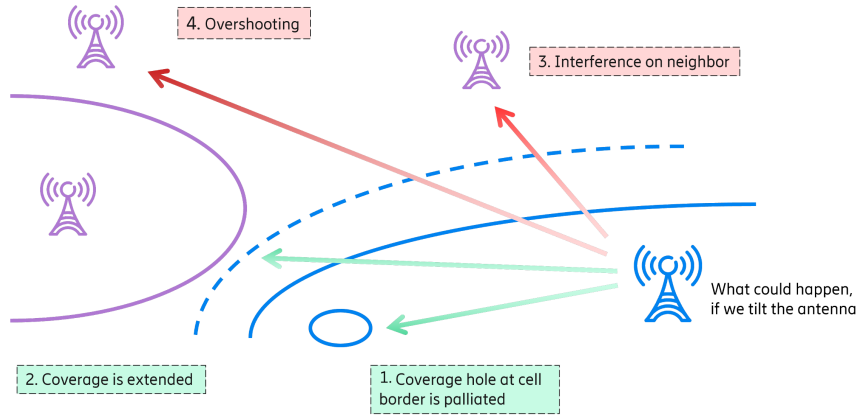


Figure 12: RET-related effects of tilt changes: (1) coverage hole at cell border, (2) coverage extension, (3) interference on neighbour, and (4) overshooting. From [28].

RET control not only a performance optimisation task, but also a challenge in safe and robust learning.

This environment thus provides a highly relevant and realistic testbed for RL. It features:

- Non-stationary dynamics, due to changing interference patterns as the agent adjusts its configuration.
- Limited observability, since the agent acts based on its local observations (AvgRSRP, AvgSINR, Tilt).
- Coupled agent interactions, as performance in one cell depends on the configuration of neighbouring cells.

3.3 Agents architecture

All agents in this thesis are based on the actor-critic framework, a class of RL methods in which two networks are trained simultaneously: an actor, which learns the policy $\pi(a | s)$, and a critic, which estimates the value function $V(s)$. We adopt the Proximal Policy Optimization algorithm as the training backbone for both our KAN-based and baseline agents as discussed in Subsection 2.2.3.

To support the use of Kolmogorov-Arnold Networks within the Proximal Policy Optimization framework, the practical implementation relies on the open source PyKan library, which implements the spline-based activation functions and handles their gradient propagation, as well as the symbolification step¹. On top of this, we implemented the KANActor and KANCritic modules, adapting the base KAN class to the PPO requirements. The PPO agent itself and its training loop were also implemented from scratch.

3.3.1 KAN-Based Agent

In the proposed architecture, both the actor and critic networks are implemented using KAN, which replace traditional MLP.

Beyond the standard PPO training procedure, KAN integrate regularisation terms designed to control the complexity and behaviour of the learned representations, as discussed in subsection 2.1.5.1. These regularisations serve both to improve generalisation and to preserve interpretability, a key advantage of the KAN framework. Specifically:

- L^1 regularisation is applied to the edge functions to promote simplification of the resulting functional graph.
- Entropy regularisation is introduced to promote less complex function shapes.
- Coefficient smoothness penalty is used to constrain the spline co-

¹Which relies itself on SymPy.

efficients of each edge function, reducing oscillatory behaviour and abrupt changes.

Together, these regularisation penalties form a crucial component of the training pipeline, guiding the network toward parsimonious and semantically meaningful solutions. Their integration within the PPO framework enables a seamless balance between expressive function approximation and structural simplicity.

3.3.2 MLP Baseline

To provide a fair baseline for comparison, we implement a PPO agent using standard MLP for both actor and critic networks. The MLP are designed to match the KAN architecture in terms of expressiveness, typically using two hidden layers with ReLU activations and the same number of hidden units per layer.

The MLP baseline does not include any symbolic or sparsity constraint, and is treated as a black-box model throughout the experiments. Its role is to serve as a performance benchmark and as a reference for evaluating the interpretability of the KAN-based agent.

3.3.3 Shared Architecture Considerations

To ensure a fair comparison between the KAN and MLP models, both architectures are trained using the same PPO hyperparameters and on-policy rollouts for the CartPole environment. The actor and critic are optimised separately using independent optimisers and learning rate schedules. Both networks are initialised with the same random seed to minimise stochastic variability in comparative results.

Regarding the RAN environment, and considering the higher complexity of the dynamics compared to CartPole, all comparisons will be based on optimised network configurations. This include tuning all the PPO hyperparameters:

- discount factor: γ ,

- GAE parameter: λ ,
- clipping parameter: ϵ ,
- value function loss coefficient: c_v ,
- entropy regularisation coefficient: β ,
- learning rate (actor and critic): lr .

For KAN-based agents, the architecture-specific parameters are also optimised, such as:

- number of splines per layer n_l ,
- number of layers: L ,
- grid size: G ,
- spline degree: k .

Regularisation hyperparameters are also tuned:

- L^1 regularization coefficient: μ_1 ,
- $\mathcal{H}$ entropy coefficient: μ_2 ,
- $\mathcal{C}$ magnitude penalty: μ_3 ,
- $\mathcal{C}_{\text{diff}}$ smoothness penalty: μ_4 ,
- overall KAN regularisation strength: λ_{KAN} ,
- pruning threshold: τ ,
- pruning interval t_{prune} , .

An equivalent tuning procedure is applied to the MLP based agent, optimising architectural hyperparameters such as the number of layers, neurons per layer, and corresponding regularisation parameters to ensure a fair comparison.

While the focus of this thesis is on the interpretability benefits of using KAN, particular care has been taken to ensure that performance comparisons remain valid.

3.4 Training

The training of KAN is carried out using the PPO framework, equipped with modifications to support sparsity and interpretability. This section outlines the practical implementation of the training loop, describes and concludes with a complete summary in algorithmic form.

3.4.1 Overview

The PPO training pipeline is structured into two main phases:

1. Learning phase: the KAN agent is trained using PPO with regularisation, having in mind sparsity and simplicity,
2. Interpretability phase: once training has converged, the components considered negligible are pruned, and the remaining ones are approximated using symbolic regression.

While the general PPO algorithm has already been introduced in Section 2.2.3, we now restate only the essential building blocks relevant to our implementation, followed by a formal procedure summary.

3.4.2 PPO Update with Regularisation

The loss function used for the PPO update step is:

$$\begin{aligned} \mathcal{L}_{\text{PPO}} = & \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \\ & - \beta \cdot \mathcal{H}(\pi_\theta) + c_v \cdot \text{MSE}(V_\theta(s_t), R_t) \end{aligned} \quad (3.3)$$

where:

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the policy ratio,
- $\hat{A}_t$ is the advantage estimate computed using GAE),
- $\mathcal{H}(\pi_\theta)$ is the policy entropy,

- $R_t = \hat{A}_t + V_\theta(s_t)$ is the target return.

When training KAN, an additional regularisation term is added:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{PPO}} + \lambda_{\text{KAN}} \cdot \mathcal{R}_{\text{KAN}}, \quad (3.4)$$

where $\mathcal{R}_{\text{KAN}}$ includes L^1 penalties on edge activations, entropy penalties on spline basis weights, and coefficient regularisation terms to encourage interpretability, as described in Equation (2.12).

Combining both, we have that our final loss is:

$$\begin{aligned} \mathcal{L}_{\text{total}} = \mathbb{E}_t \bigg[& \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \\ & - \beta \cdot \mathcal{H}(\pi_\theta) + c_v \cdot \text{MSE}(V_\theta(s_t), R_t) \\ & + \lambda_{\text{KAN}} \sum_l (\mu_1 \|\Phi_l\|_1 + \mu_2 \mathcal{H}(\Phi_l) + \mu_3 \mathcal{C}(\Phi_l) + \mu_4 \mathcal{C}_{\text{diff}}(\Phi_l)) \bigg]. \end{aligned} \quad (3.5)$$

3.4.3 Full Training Algorithm

Algorithm 1 describes the entire training workflow, from PPO optimisation to interpretability enhancements. In Chapter 4, we demonstrate how this pipeline leads to highly compact, symbolic policies with minimal performance degradation.

The full training procedure outlined in Algorithm 1 strongly resemble the workflow described in Subsection 2.1.6. Figure 4 summarises the central philosophy of the approach: the model is first trained with regularisation constraints (Step 1), followed by filtering out inactive functions (Step 2). Then replacement of active functions with symbolic one is performed (Step 3), with a subsequent fine-tuning of affine parameters (Step 4), ultimately yielding a fully symbolic, interpretable model (Step 5 and Step 6).

This structured framework deliver a clear step-by-step recipe to obtain a interpretable model starting from a black-box one, which is reflected in our PPO training pipeline.

Algorithm 1 KAN-PPO Training Loop

Require: Environment $\mathcal{E}$, actor network π_θ , critic network V_ϕ , buffer $\mathcal{B}$, total episodes N

Require: PPO hyperparameters $(\gamma, \lambda, \epsilon, \beta, c_v)$, pruning interval K , pruning threshold τ

- 1: Initialize parameters θ, ϕ
- 2: **for** episode = 1 to N **do**
- 3: Reset environment: $s_0 \leftarrow \mathcal{E}.\text{reset}()$
- 4: **while** episode not done **do**
- 5: Sample action $a_t \sim \pi_\theta(\cdot \mid s_t)$
- 6: Observe reward r_t , next state s_{t+1} , and done flag
- 7: Store $(s_t, a_t, r_t, \pi_\theta(a_t \mid s_t), V_\phi(s_t))$ in buffer $\mathcal{B}$
- 8: $s_t \leftarrow s_{t+1}$
- 9: **end while**
- 10: **if** buffer contains at least T transitions **then**
- 11: Compute advantages and returns using GAE:
 $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
 $\hat{A}_t = \delta_t + \gamma \lambda \hat{A}_{t+1}$
 $R_t = \hat{A}_t + V_\phi(s_t)$
- 12: **for** each PPO epoch **do**
- 13: **for** each minibatch **do**
- 14: $r_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$
- 15: $L_\pi = -\min \left(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right)$
- 16: $L_V = (V_\phi(s_t) - R_t)^2$
- 17: Compute policy entropy $\mathcal{H}(\pi_\theta)$
- 18: Compute KAN regularization term $\mathcal{R}_{\text{KAN}}$
- 19: $\mathcal{L}_{\text{total}} = L_\pi + c_v \cdot L_V - \beta \cdot \mathcal{H} + \lambda_{\text{KAN}} \cdot \mathcal{R}_{\text{KAN}}$
- 20: Update θ, ϕ using gradient descent on $\mathcal{L}_{\text{total}}$
- 21: **end for**
- 22: **end for**
- 23: Clear buffer $\mathcal{B}$
- 24: **end if**
- 25: **if** episode mod $K = 0$ **then**
- 26: **Prune** all spline functions φ_{ij} with $\|\varphi_{ij}\|_1 < \tau$
- 27: **end if**
- 28: **Symbolify** all active spline functions via symbolic regression
- 29: **Continue training** using symbolic networks, optimising only the remaining affine coefficients

4 Results

In this chapter, we present the evaluation of the proposed method, combining Kolmogorov-Arnold Networks with Proximal Policy Optimization. The evaluation starts first with the standard CartPole environment, and later on a real-world-inspired Radio Access Networks (RAN) scenario.

The goal of this assessment is threefold:

- i.* to assess the training performance of KAN-based policies compared to standard MLP,
- ii.* to quantify the degree of network sparsity and parameter efficiency achieved through KAN,
- iii.* to demonstrate how symbolic interpretability can be obtained from the trained agent’s policy.

4.1 Experiments on the CartPole Environment

To compare performance and interpretability, we consider two different PPO architectures: a standard MLP baseline and a KAN. Both models are trained using the same PPO algorithm described in Chapter 3, with identical hyperparameters, training schedules and seeds for random generation.

PPO Hyperparameters

Parameter	Value
Discount factor γ	0.99
GAE parameter λ	0.95
Clipping parameter ϵ	0.2
Value function loss coefficient c_v	0.5
Entropy regularization coefficient β	0.01
Learning rate (actor and critic)	$3 \cdot 10^{-4}$
Steps per rollout	2048
PPO epochs per update	8
Mini-batch size	64
Seeds	42, 43, 44, 45, 46

Table 1: Common PPO hyperparameters used during CartPole experiments.

Network Architectures

Multi-Layer Perceptrons

The MLP baseline architecture follows a standard two-layer feedforward design with ReLU activations.

Component	Hidden Layers	Activation	Parameters
Actor MLP	2×32 units	ReLU	1,282
Critic MLP	2×32 units	ReLU	1,249

Table 2: Architecture details for the MLP actor and critic networks for CartPole.

Kolmogorov-Arnold Networks

The KAN architecture is designed with interpretability in mind. The grid size and spline degree are fixed across all experiments. The architecture is described in Table 3.

Component	Layers	Splines	Grid Size	Degree	Parameters
Actor KAN	1	2	5	3	112
Critic KAN	1	1	5	3	56

Table 3: Architecture details for the KAN actor and critic networks for the CartPole experiments.

Table 4 reports the values used for the regularisation terms, as well as the pruning schedule employed during training.

Parameter	Value
L^1 regularization coefficient μ_1	1.0
$\mathcal{H}$ entropy coefficient μ_2	1.0
$\mathcal{C}$ coefficient magnitude penalty: μ_3	1.0
$\mathcal{C}_{\text{diff}}$ smoothness coefficient penalty μ_4	1.0
Overall KAN regularization strength λ_{KAN}	0.01
Pruning threshold τ	0.05
Pruning interval (episodes)	500

Table 4: KAN-specific regularisation and pruning parameters for CartPole.

Results

Reward Curves and Training Time

To evaluate the learning efficiency and overall performance of the models, we compare cumulative rewards and training durations for MLP and KAN.

Figure 13 shows the learning curves for both agents. The MLP agent converges within the first 500 episodes, while the KAN agent converges more gradually with slightly higher variance.

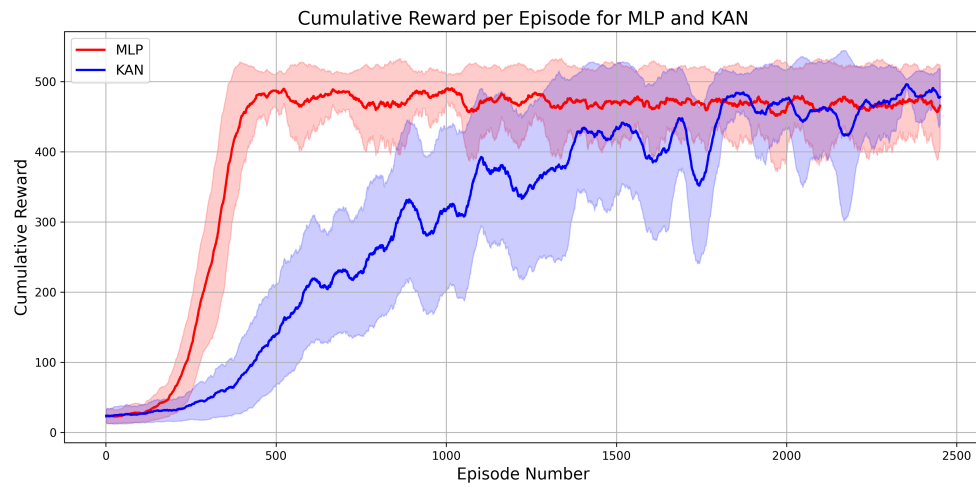


Figure 13: Cumulative reward per episode for KAN (blue) and MLP (red). Curves are smoothed using a 50-episode moving average. Shaded areas show standard deviation across five seeds.

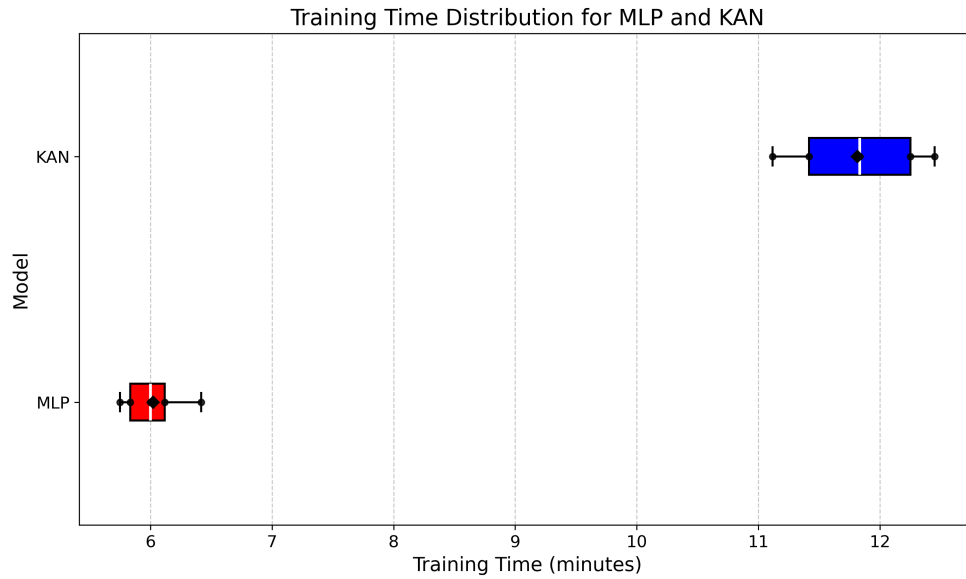


Figure 14: Training time distribution (in minutes) for MLP (red) and KAN (blue) across five random seeds.

Model	Mean Time (min)	Std. Deviation (min)
MLP	6.01	0.14
KAN	11.49	0.30

Table 5: Mean and standard deviation of training time across five runs.

Figure 14 and Table 13 report the distribution and statistics of training times across five seeds.

Model Snapshots and Symbolic Policy

The final symbolic expressions for the actor are shown in Equation 4.1, and the corresponding decision boundary is visualised in Figure 15.

$$\begin{cases} a_{\text{left}} = -2.44 \tanh(4.12\theta - 0.09) - 1.68 \tanh(1.3\dot{\theta} + 0.03) \\ a_{\text{right}} = 2.56 \tanh(4.1\theta - 0.1) + 2.03 \tanh(1.1\dot{\theta} + 0.06) + 0.17 \\ a^* = \arg \max\{a_{\text{left}}, a_{\text{right}}\} \end{cases}$$

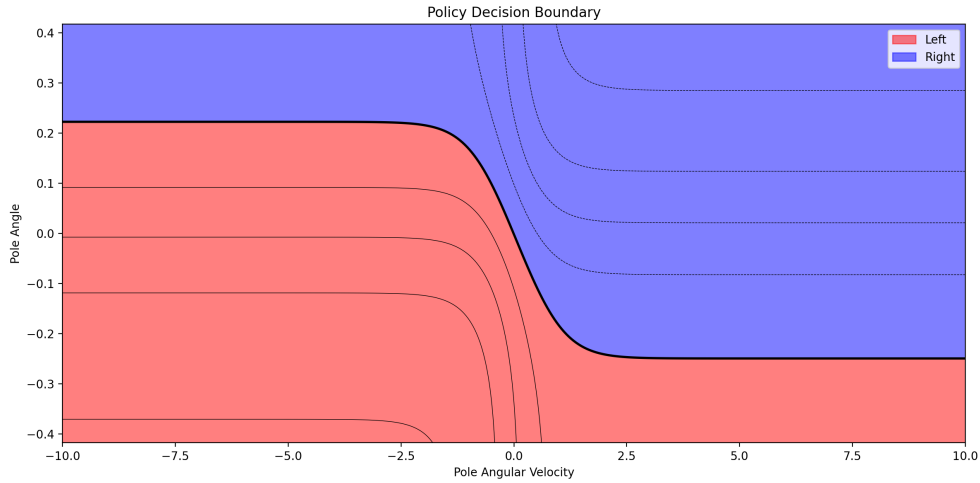


Figure 15: Symbolic policy formula and decision boundary in the $(\theta, \dot{\theta})$ plane.

Figures 16-19 show the KAN actor model throughout different training phases: initialisation, training, pruning, and symbolic distillation.

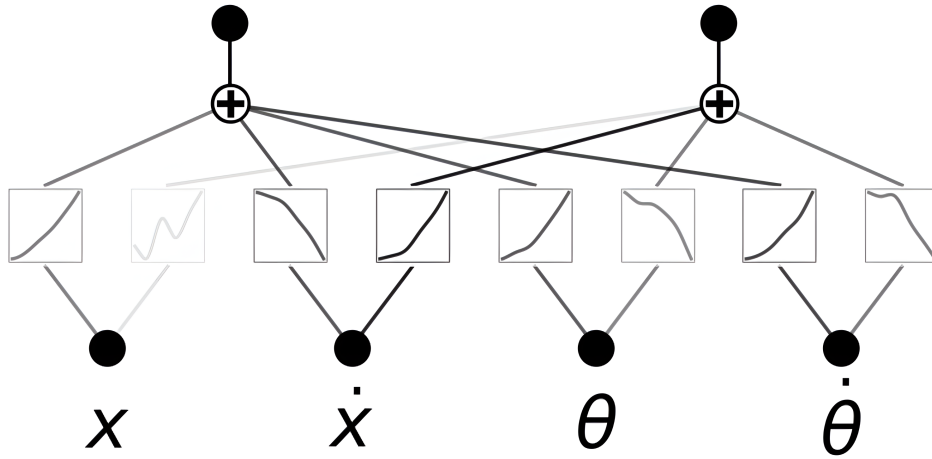


Figure 16: Initial dense KAN at initialisation, with randomly initialised splines. All input variables are connected.

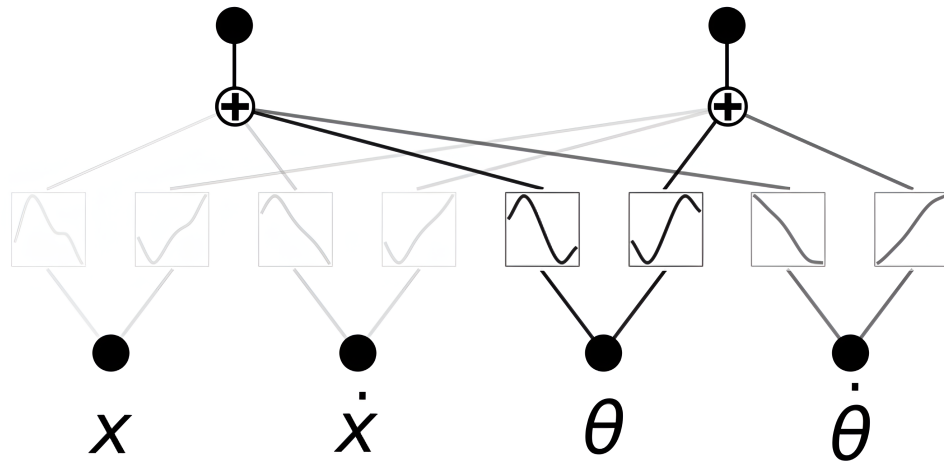


Figure 17: KAN after training with sparsity-inducing regularisation. The model has begun deactivating unnecessary edges and splines.

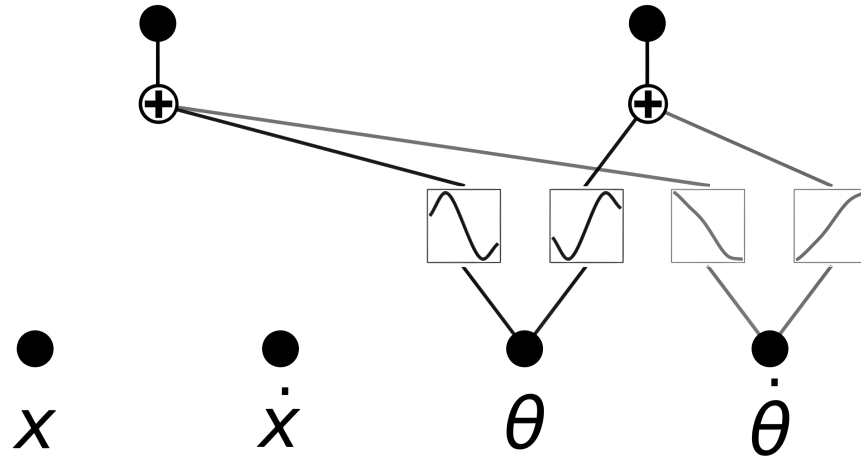


Figure 18: Post-pruning: the network retains only a subset of edges. Notably, inputs x and $\dot{x}$ are no longer used, as the model learns to rely on θ and $\dot{\theta}$.

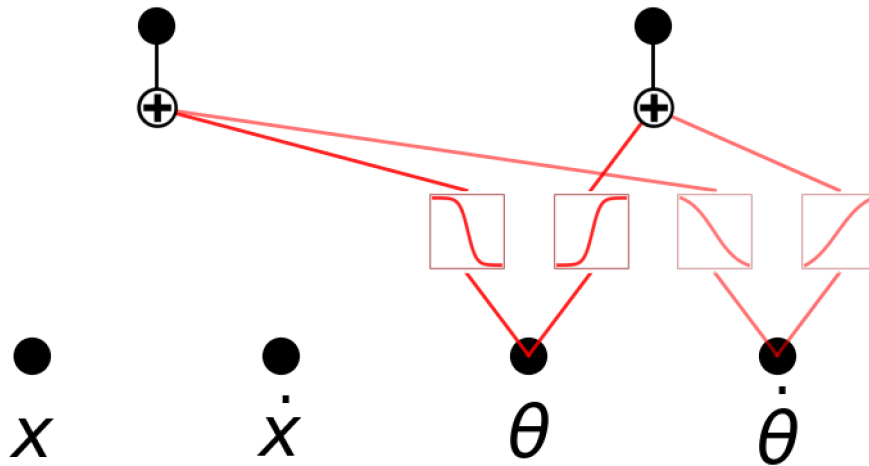


Figure 19: Symbolified KAN model. Red spline functions have been replaced by closed-form expressions.

4.2 Experiments on the RAN Environment

To assess model interpretability in a more complex setting, we repeat the same evaluation procedure on the custom RET environment. Both MLP and KAN are trained with identical PPO hyperparameters, summarised in Table 6. The maximum episode length is fixed at 20 steps.

PPO Hyperparameters

Parameter	Value
Discount factor γ	0.99
GAE parameter λ	0.95
Clipping parameter ϵ	0.2
Value-function loss coefficient c_v	0.5
Entropy regularisation coefficient β	0.01
Learning rate (actor & critic)	$1 \cdot 10^{-3}$
Steps per rollout	2 000
PPO epochs per update	6
Mini-batch size	64

Table 6: Common PPO hyperparameters used during RET experiments.

Network Architectures

Multi-Layer Perceptrons

MLP serve as baselines. Actor and critic share the same architecture and are trained for 5,000 episodes. The tested configurations are listed in Table 7.

Actor Architecture	Critic Architecture
[32, 32]	[32, 32]
[64, 64]	[64, 64]
[128, 128]	[128, 128]

Table 7: MLP architectures used for the RAN environment.

Kolmogorov–Arnold Networks

For KAN, architecture and regularisation hyperparameters were explored via a coarse random search¹. All models were trained for 5,000 episodes; pruning was performed every 1,000 episodes with threshold $\tau = 0.01$. Based on reward, the best configuration was fine-tuned for an additional 1,000 episodes under symbolic constraints.

Regularisation Parameter	Values Tested
L^1 coefficient μ_1	[0.0, 1.0, 2.0]
Entropy coefficient μ_2	[0.0, 1.0, 2.0]
Magnitude penalty μ_3	[0, 0.2, 0.5]
Smoothness penalty μ_4	[0, 0.2, 0.5]

Table 8: KAN regularisation hyperparameters tested in the RAN environment.

Architecture Parameter	Values Tested
Actor grid size	[5, 8]
Actor spline degree	[3, 5]
Actor # hidden layer functions	[0, 3, 5]
Critic grid size	[5, 8]
Critic spline degree	[3, 5]
Critic # hidden layer functions	[0, 3, 5]

Table 9: KAN actor and critic architecture settings explored in the RAN environment.

Selected Architectures

After the hyperparameter search was completed, the best-performing configurations, reward-wise, were selected for both KAN and MLP agents. To test the consistency and robustness of the obtained outcomes, each configuration was then retrained using five different random seeds, namely 42, 43, 44, 45, 46.

This experimental setup allowed for the assessment of the stability of the learning procedures under different initial conditions. The final hyperparameter settings and corresponding parameter counts are summarised below.

¹Exhaustive grid search was computationally infeasible.

Component	Architecture
Actor	[64, 64]
Critic	[64, 64]

Table 10: Best MLP architecture (baseline).

Parameter	Value	Parameter	Value
L^1 penalty μ_1	2.0	L^1 penalty μ_1	0.0
Entropy penalty μ_2	1.0	Entropy penalty μ_2	0.0
Magnitude penalty μ_3	0.2	Magnitude penalty μ_3	0.0
Smoothness penalty μ_4	0.2	Smoothness penalty μ_4	0.0
Grid size	5	Grid size	5
Splines degree k	3	Splines degree k	3
Actor # hidden layer functions	0	Critic # hidden layer functions	0

(a) Actor.
(b) Critic.

Table 11: Best KAN hyperparameters for the RAN environment.

Component	# Params	Component	# Params
Actor KAN	126	Actor MLP	4 611
Critic KAN	42	Critic MLP	4 481

(a) KAN agent.
(b) MLP baseline.

Table 12: Trainable parameter counts for the selected models.

Reward Curves and Training Time

To evaluate the learning efficiency and overall performance of the models, we compare cumulative rewards and training durations for MLP and KAN.

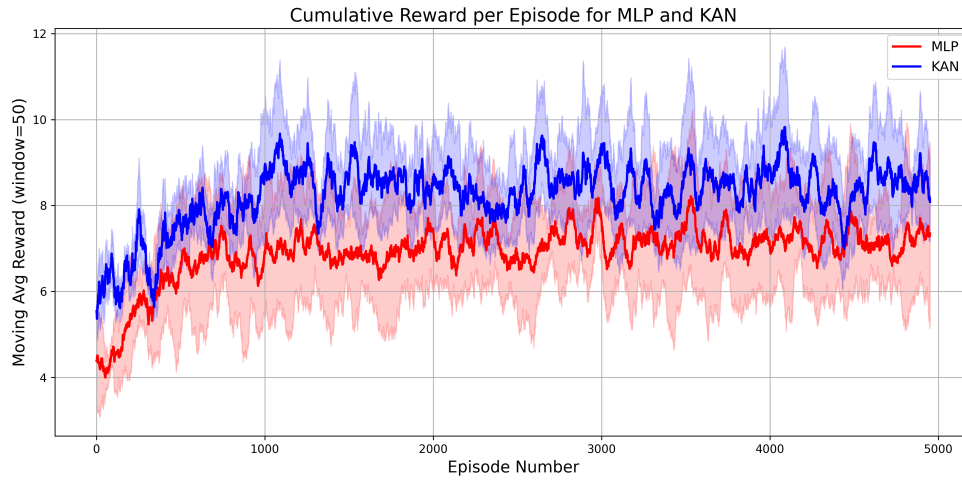


Figure 20: Cumulative reward per episode for KAN (blue) and MLP (red). Curves are smoothed using a 50-episode moving average. Shaded areas show standard deviation across five seeds.

Figure 20 shows the learning curves for both agents. While overall trends are similar, KAN converge to a slightly better policy than MLP and exhibit lower variance during training.

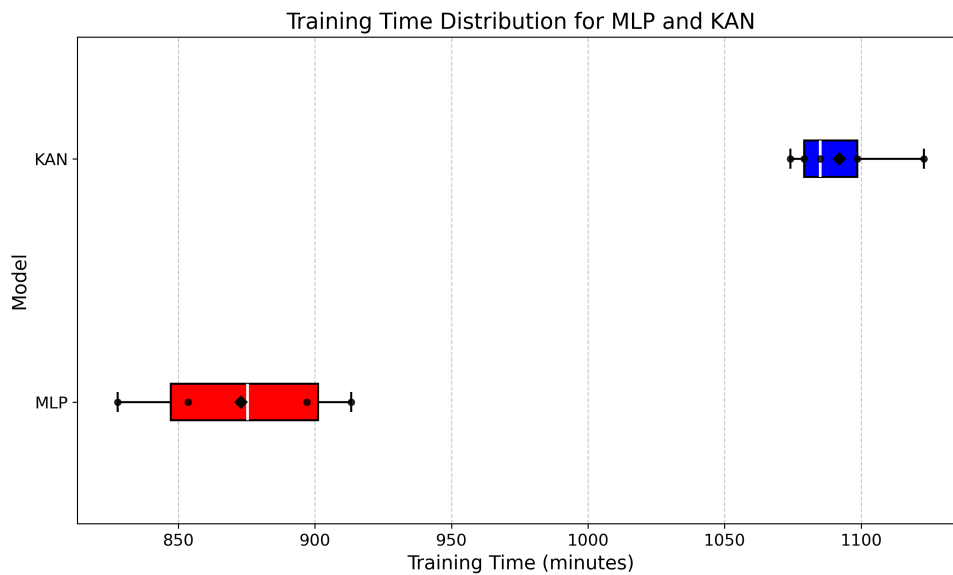


Figure 21: Training time distribution (in minutes) for MLP (red) and KAN (blue) across five random seeds.

Model	Mean Time (min)	Std. Deviation (min)
MLP	872.34	33.54
KAN	1091.45	17.28

Table 13: Mean and standard deviation of training time across five runs.

Figure 14 and Table 13 report the distribution and statistics of training times across five seeds.

Figure 22 shows the learning curves for the KAN agent before and after symbolification. Performance remains stable across the transition, with both curves exhibiting similar cumulative rewards and variance. This suggests that the symbolic policy retains most of the performance characteristics of the original network.

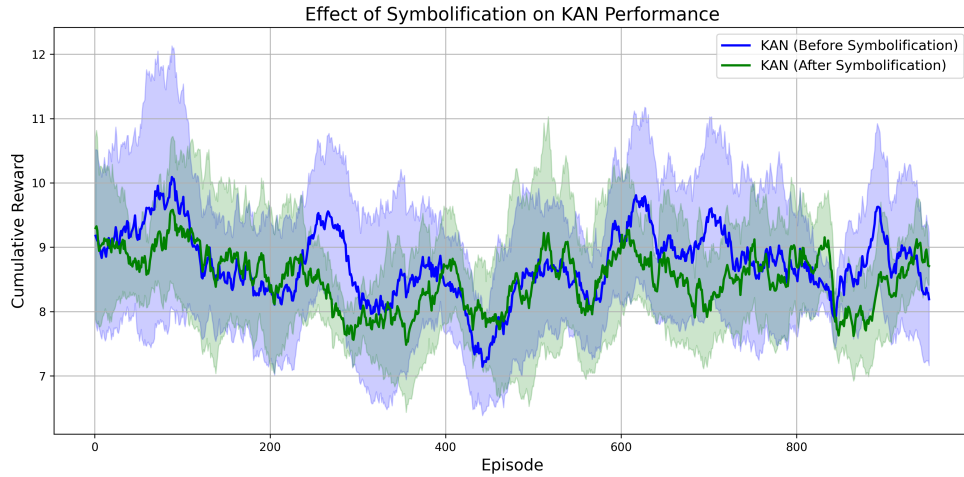


Figure 22: Cumulative reward per episode for KAN before (blue) and after (green) symbolification. Curves are smoothed using a 50-episode moving average. Shaded areas show standard deviation across five seeds.

Figure 23 displays the smoothed reward distributions obtained from evaluation over the same 1,000 episodes across five training seeds. Both KAN and MLP agents achieve similar average performance in terms of mean reward. However, the lower standard deviation in the KAN distribution suggests more consistent policy behaviour.

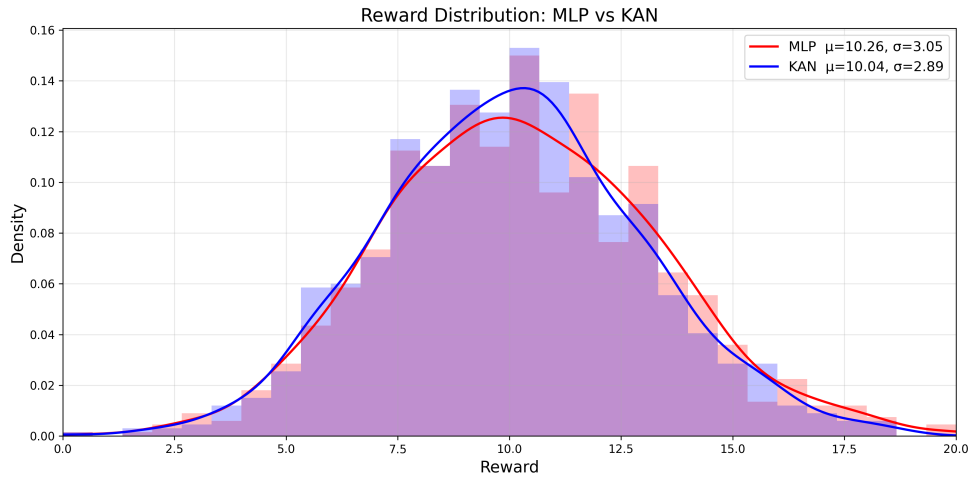


Figure 23: Reward distribution comparison between MLP and KAN agents in the RAN environment.

Model	Mean Reward	Std. Deviation
MLP	10.26	3.05
KAN	10.04	2.89

Table 14: Mean and standard deviation of rewards across evaluation episodes.

Model Snapshots and Symbolic Policy

The final symbolic expressions for the actor are shown in Equation 4.1, and the corresponding decision boundary is visualised in Figure 25.

$$\left\{ \begin{array}{l} a_{-1^\circ} = 0.05 \cdot \text{ReLU}(4.41 \cdot \text{SINR} - 0.21) \\ \quad + 0.06 \cdot \text{ReLU}(5.47 \cdot \text{RSRP} + 3.03) \\ \quad + 0.07 \cdot |7.23 \cdot \theta + 3.01| - 0.72 \\ \\ a_{+0^\circ} = 0.02 \cdot \text{ReLU}(9.97 \cdot \text{SINR} - 0.61) \\ \quad + 0.14 \cdot \text{ReLU}(4.48 \cdot \text{RSRP} + 0.4) - 0.23 \\ \\ a_{+1^\circ} = 0.05 \cdot \text{ReLU}(3.49 \cdot \text{SINR} + 1.84) \\ \quad - 0.11 \cdot \text{ReLU}(7.41 \cdot \text{RSRP} + 6.21) \\ \quad - 0.13 \cdot |5.95 \cdot \theta + 2.05| + 1.67 \\ \\ a^* = \arg \max \{a_{-1^\circ}, a_{+0^\circ}, a_{+1^\circ}\} \end{array} \right. \quad (4.1)$$

where the function $\text{ReLU}(x)$ returns zero for negative inputs and behaves linearly for positive inputs. Its behaviour is shown in Figure 24.

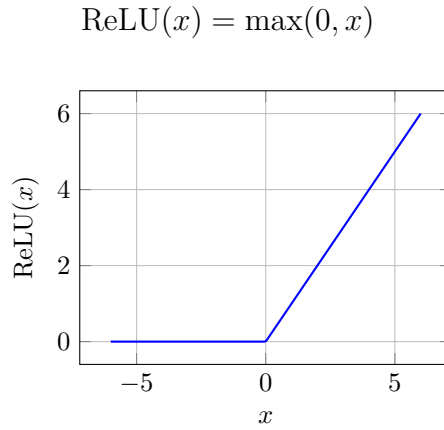


Figure 24: The ReLU (Rectified Linear Unit) function.

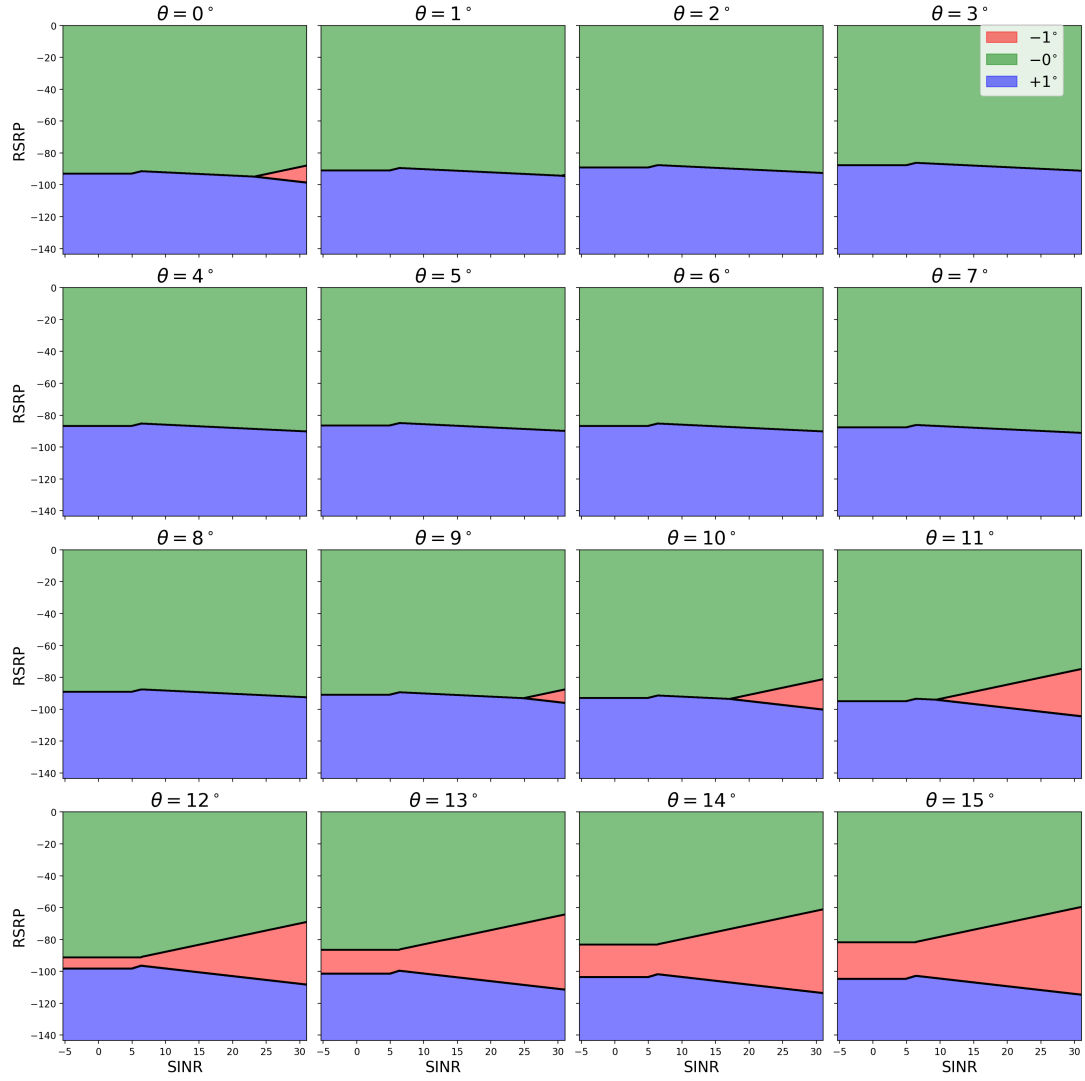


Figure 25: Symbolic policy formula and decision boundary in the (SINR, RSRP) plane for each value of the tilt angle θ .

Figures 26-28 show the KAN actor model throughout different training phases: initialisation, training, pruning, and symbolic distillation.

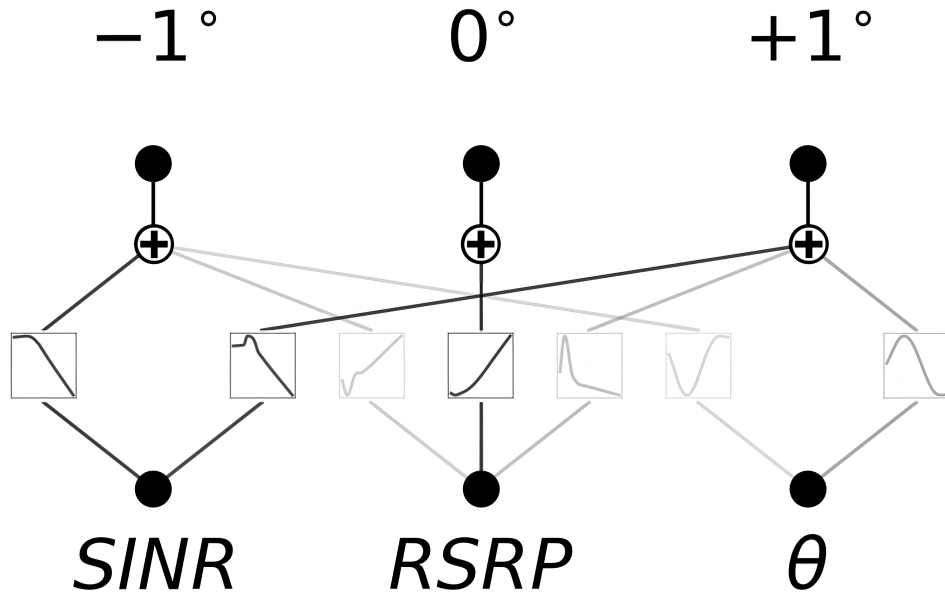


Figure 26: Initial dense KAN at initialisation, with randomly initialised splines. All input variables are connected.

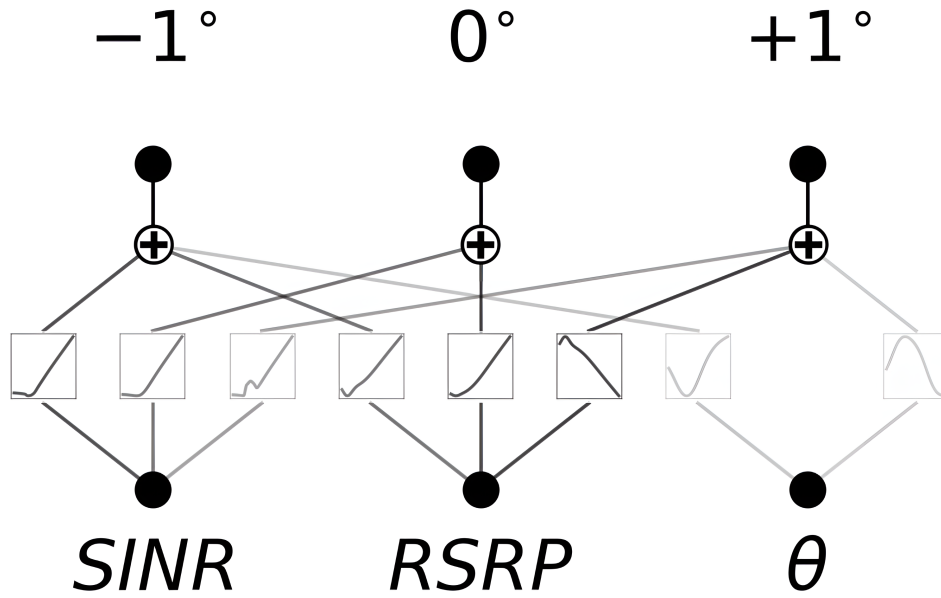


Figure 27: KAN after training with sparsity-inducing regularisation. The model has begun deactivating unnecessary edges and splines.

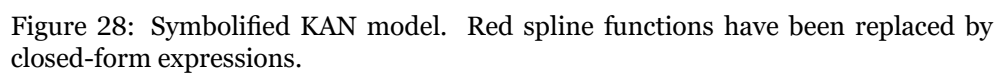


Figure 28: Symbolified KAN model. Red spline functions have been replaced by closed-form expressions.

5 Discussion

In this chapter, we interpret and contextualise the results presented in Chapter 4. We examine the observed learning behaviours, model structures, and symbolic outcomes across the evaluated environments. The discussion is organised per environment, starting with the classical CartPole control task, followed by the RAN scenario.

5.1 Discussion on the CartPole Environment

Training Efficiency and Learning Dynamics

The MLP agent reaches high performance quickly, while the KAN agent exhibits slower convergence. This delay is expected, as KAN models are trained with multiple regularisation terms that inhibit early aggressive updates. Furthermore, spline evaluations are computationally more intensive than matrix multiplications, and the PyKan library lacks the low-level optimisation found in PyTorch’s MLP backend.

Despite slower training and longer wall-clock time (Figure 14), the KAN agent ultimately reaches competitive performance. This highlights a key trade-off: MLPs offer training speed and simplicity, whereas KANs introduce interpretability and built-in feature selection, at a *modest* computational cost.

Sparsity and Symbolic Structure

The pruning phase (Figure 18) reveals that the agent learns to ignore the cart position x and velocity $\dot{x}$, relying only on θ and $\dot{\theta}$. This aligns with physical intuition: balancing the pole depends more directly on

its angular state than on the cart’s position.

The symbolic expressions in Equation 4.1 offer a transparent, interpretable description of the learned policy. The smooth $\tanh$ -based terms match the continuous control nature of the task. The decision boundary (Figure 15) is symmetric and non-linear, illustrating that the agent respects the system’s structure while introducing nonlinear corrections to optimise behavior.

This symbolic output provides a human-readable alternative to black-box neural policies, supporting applications where transparency, verifiability, or safety constraints are crucial.

5.2 Discussion on the RAN Environment

Training Efficiency and Learning Dynamics

The training dynamics observed in the RAN environment show similar behaviour between KAN and MLP agents in terms of rewards and final performances, Figure 20 and Figure 23 respectively. The main differences can be seen in the number of parameters used, Table 6, where KAN had ≈ 40 times fewer parameters, but needed $\approx 25\%$ more training time, Table 21.

The complexity of the spine-based activation functions primarily introduces this training overhead. This overhead is reflected in the training time distribution, Figure 21, where KAN showed a consistent increase in computational time compared to MLP.

Despite this initial overhead, KAN agents showed robust and stable learning dynamics, reflected in a consistent improvement and then plateau, of training rewards, Figure 20. By contrast, MLP showed a more volatile training behaviour.

An essential step in the training loop was the symbolification process, Figure 22, where a symbolic expression replaced the spline-based functions. This phase resulted in a minimal performance degradation, demonstrating that the symbolic approximation accurately captured

the decision-making behaviour learned by the network. Moreover, the shift to symbolic representation substantially enhanced interpretability, without sacrificing performance.

Sparsity and Symbolic Structure

An unexpected outcome emerged during the hyperparameter tuning phase: the best-performing KAN model was the smallest, featuring no hidden layers, and involving just 112 trainable parameters. As a result, the potential for sparsification was limited due to the already minimal architecture. Despite this, some pruning still occurred, as illustrated in Figure 27.

The resulting symbolic structure was simplified, reflecting a direct and precise mapping between input variables and actions. This minimalistic symbolic structure highlights the key idea of KAN, showcasing their ability to learn minimal and meaningful relationships that not only domain experts can interpret and understand. The simplicity and clarity of the final symbolic expressions reinforce their value in deployment scenarios where interpretability, trust, and ease of analysis are essential.

Interpretation of the Policy in a Physical Context

The final policy, as shown in Equation 4.1 and plotted in Figure 25, provides an explicit and interpretable mapping between the two network performance metrics, Signal to Interference plus Noise Ratio and Reference Signal Received Power, and the corresponding tilt actions. Understanding these metrics in their physical terms is essential to appreciate the rationale behind the policy behaviour.

SINR refers to the quality of the received signal. A high SINR means a strong signal, typically requiring little to no adjustment. Conversely, low SINR indicates a noisy environment, where interference impacts signal quality, calling for adjustments to antenna tilt angle.

RSRP refers instead to the strength of the signal measured at the

UEs. Higher RSRP values suggest a stable and strong signal reception, typically requiring minimal or no adjustments. Lower RSRP values indicate a weaker signal, resulting in a poor coverage or connectivity issue, demanding tilt adjustments.

The policy derived from the chosen KAN model, Equation 4.1, aligns with these physical intuitions. As shown in Figure 25, region with lower SINR and/or lower RSRP initiate explicit tilt adjustments of $\pm 1^\circ$. By contrast, areas characterised by higher SINR and RSRP require no adjustments, reflecting the optimal conditions of the environment.

Beyond the policy behaviour, this workflow is a compelling proof of concept. It demonstrates model performance and interpretability, strengthening confidence in its applicability to real-world Radio Access Networks.

6 Conclusion

This chapter addresses each research question posed in the Introduction, summarising the contributions and key findings of the thesis.

RQ1: Can Kolmogorov-Arnold Networks be effectively integrated into policy and value networks for Reinforcement Learning?

Yes. This thesis demonstrated that Kolmogorov-Arnold Networks can be effectively integrated into the Proximal Policy Optimization framework as both actor and critic networks.

KAN were successfully trained in the CartPole and RAN environments. In both cases, KAN based agents learned policies that matched or exceeded the performance of the corresponding MLP based baselines. Regularisation enabled the KAN to converge to sparse representations without compromising convergence stability or learning dynamics.

RQ2: Do Kolmogorov-Arnold Network-based Reinforcement Learning agents achieve competitive performance compared to conventional MLP-based agents in realistic network scenarios such as Remote Electrical Tilt?

The experimental results demonstrate that KAN agents can achieve competitive performance compared to conventional MLP agents. In both the CartPole and RAN environments, the best KAN models reached final performance levels comparable to those of the strongest MLP baselines, despite using $\approx \times 40$ less parameters.

For instance, in CartPole, a KAN actor with only 126 trainable pa-

rameters achieved the same cumulative reward as an MLP actor with 8385 parameters. In the RAN environment, similar trends were observed: the selected KAN configuration matched the MLP baseline in terms of final reward while offering a substantially smaller and sparser model.

These results confirm that KAN can serve as lightweight and performant alternatives to MLP in both benchmark and realistic RL tasks.

RQ3: To what extent do Kolmogorov-Arnold Networks enhance the interpretability of Reinforcement Learning policies?

One of KAN’s main advantages is their ability to yield interpretable representations. This thesis assessed interpretability primarily through symbolic policy extraction and decision boundary visualisation.

After regularised training and pruning, KAN policies could be expressed as compact symbolic formulas involving only a subset of the original input features. These formulas provided transparent, human-readable descriptions of the decision logic. In CartPole and RAN, the symbolic expressions reflected meaningful control behaviour and offered clear insight into which variables the agent relied on.

Furthermore, decision boundary visualisations confirmed that the symbolic policies captured the essential structure of the learned action space. Compared to conventional MLP, which act as black boxes, KAN offer a significant improvement in policy interpretability, especially when sparsity and symbolic extraction are properly enforced during training.

In addition, KAN enable snapshot inspection of the model throughout training, showing the evolution of the learned representations. This introspection reveals how the network gradually refines its focus, discarding irrelevant inputs while retaining salient features. Such transparency into the learning dynamics is typically inaccessible in standard

architectures and provides a unique opportunity to understand and debug the model’s internal mechanisms.

RQ4: What computational trade-offs are introduced by Kolmogorov-Arnold Networks, and are they acceptable in practice?

KANs introduce moderate computational overhead compared to conventional MLPs, primarily due to the spline-based edge functions and regularisation terms used during training. In both the CartPole and RAN environments, KANs agents required longer training times per episode and per update cycle.

Despite these costs, the training remained stable and converged reliably across different seeds and configurations. The benefits of increased interpretability and reduced model size may offset the added complexity in some particular use cases. In particular, KANs achieved similar or better performance than MLPs while using an order of magnitude fewer parameters.

Overall, the computational trade-offs introduced by KANs may be acceptable in domains where model transparency and compactness are critical design requirements.

Bibliography

- [1] Muhammad K. Shehzad et al. “Artificial Intelligence for 6G Networks: Technology Advancement and Standardization”. In: *IEEE Vehicular Technology Magazine* 17.3 (Sept. 2022), pp. 16–25. ISSN: 1556-6080. DOI: 10.1109/mvt.2022.3164758. URL: <http://dx.doi.org/10.1109/MVT.2022.3164758>.
- [2] Huihan Yu et al. “Deep Reinforcement Learning Based Beamforming for Throughput Maximization in Ultra-Dense Networks”. In: *2022 IEEE Wireless Communications and Networking Conference (WCNC)*. 2022, pp. 1021–1026. DOI: 10.1109/WCNC51071.2022.9771887.
- [3] Filippo Vannella et al. “Remote Electrical Tilt Optimization via Safe Reinforcement Learning”. In: *2021 IEEE Wireless Communications and Networking Conference (WCNC)*. 2021, pp. 1–7. DOI: 10.1109/WCNC49053.2021.9417363.
- [4] Erika Puiutta and Eric MSP Veith. *Explainable Reinforcement Learning: A Survey*. 2020. arXiv: 2005.06247 [cs.LG]. URL: <https://arxiv.org/abs/2005.06247>.
- [5] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5 (1989), pp. 359–366. ISSN: 0893-6080. DOI: [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8). URL: <https://www.sciencedirect.com/science/article/pii/0893608089900208>.
- [6] Simon Haykin. *Neural Networks: A Comprehensive Foundation*. 1st. USA: Prentice Hall PTR, 1994. ISBN: 0023527617.
- [7] Ziming Liu et al. *KAN: Kolmogorov-Arnold Networks*. 2025. arXiv: 2404.19756 [cs.LG]. URL: <https://arxiv.org/abs/2404.19756>.
- [8] Vladimir I. Arnold. “On the Representation of Functions of Several Variables as a Superposition of Functions of a Smaller Number of Variables”. In: *Collected Works: Representations of Functions, Celestial Mechanics and KAM Theory, 1957–1965*. Ed. by Alexander B. Givental et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 25–46. ISBN: 978-3-642-01742-1. DOI: 10.1007/978-3-642-01742-1_5.

- [9] Federico Girosi and Tomaso Poggio. “Representation Properties of Networks: Kolmogorov’s Theorem Is Irrelevant”. In: *Neural Computation* 1.4 (Dec. 1989), pp. 465–469. ISSN: 0899-7667. DOI: 10.1162/neco.1989.1.4.465. eprint: <https://direct.mit.edu/neco/article-pdf/1/4/465/811923/neco.1989.1.4.465.pdf>. URL: <https://doi.org/10.1162/neco.1989.1.4.465>.
- [10] Andrea Apicella et al. “A survey on modern trainable activation functions”. In: *Neural Networks* 138 (June 2021), pp. 14–32. ISSN: 0893-6080. DOI: 10.1016/j.neunet.2021.01.026. URL: <http://dx.doi.org/10.1016/j.neunet.2021.01.026>.
- [11] George V. Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of Control, Signals and Systems* 2 (1989), pp. 303–314. URL: <https://api.semanticscholar.org/CorpusID:3958369>.
- [12] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (1986), pp. 533–536. ISSN: 1476-4687. DOI: 10.1038/323533a0. URL: <https://doi.org/10.1038/323533a0>.
- [13] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second. The MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- [14] Chen Tang et al. *Deep Reinforcement Learning for Robotics: A Survey of Real-World Successes*. 2024. arXiv: 2408.03539 [cs.R0]. URL: <https://arxiv.org/abs/2408.03539>.
- [15] Volodymyr Mnih et al. *Playing Atari with Deep Reinforcement Learning*. 2013. arXiv: 1312.5602 [cs.LG]. URL: <https://arxiv.org/abs/1312.5602>.
- [16] Pradnya Kashikar. “Literature review of implementation of machine learning algorithms for improving the network security”. In: *Research Gate* Vol. 05, (Mar. 2024), pp. 6915–6922.
- [17] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- [18] Ronald J. Williams. “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning”. In: *Mach. Learn.* 8.3–4 (May 1992), pp. 229–256. ISSN: 0885-6125. DOI: 10.1007/BF00992696. URL: <https://doi.org/10.1007/BF00992696>.
- [19] John Schulman et al. *Trust Region Policy Optimization*. 2017. arXiv: 1502.05477 [cs.LG]. URL: <https://arxiv.org/abs/1502.05477>.

- [20] John Schulman et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. 2018. arXiv: 1506.02438 [cs.LG]. URL: <https://arxiv.org/abs/1506.02438>.
- [21] Scott Lundberg and Su-In Lee. *A Unified Approach to Interpreting Model Predictions*. 2017. arXiv: 1705.07874 [cs.AI]. URL: <https://arxiv.org/abs/1705.07874>.
- [22] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "Why Should I Trust You?": *Explaining the Predictions of Any Classifier*. 2016. arXiv: 1602.04938 [cs.LG]. URL: <https://arxiv.org/abs/1602.04938>.
- [23] Stephanie Milani et al. "Explainable Reinforcement Learning: A Survey and Comparative Review". In: *ACM Comput. Surv.* 56.7 (Apr. 2024). ISSN: 0360-0300. DOI: 10.1145/3616864. URL: <https://doi.org/10.1145/3616864>.
- [24] Victor Augusto Kich et al. *Kolmogorov-Arnold Network for Online Reinforcement Learning*. 2024. arXiv: 2408.04841 [cs.LG]. URL: <https://arxiv.org/abs/2408.04841>.
- [25] Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. "Neuronlike adaptive elements that can solve difficult learning control problems". In: *IEEE Transactions on Systems, Man, and Cybernetics* SMC-13.5 (1983), pp. 834–846. DOI: 10.1109/TSMC.1983.6313077.
- [26] Aleksandar Haber. *Cart Pole Control Environment in OpenAI Gym (Gymnasium) – Introduction to OpenAI Gym*. Accessed: 2025-05-11. 2023. URL: <https://aleksandarhaber.com/cart-pole-control-environment-in-openai-gym-gymnasium-introduction-to-openai-gym/>.
- [27] Franco Ruggeri et al. "Evaluation of Intrinsic Explainable Reinforcement Learning in Remote Electrical Tilt Optimization". In: *Proceedings of Eighth International Congress on Information and Communication Technology*. Ed. by Xin-She Yang et al. Singapore: Springer Nature Singapore, 2024, pp. 835–854. ISBN: 978-981-99-3236-8.
- [28] Ericsson. *How a real 5G quantum AI use case could disrupt antenna tilting*. Accessed: 2025-05-08. Sept. 2023. URL: <https://www.ericsson.com/en/blog/2023/9/how-a-real-5g-quantum-ai-use-case-could-disrupt-antenna-tilting>.